



Bachelorarbeit

Natural Language Web Interaction

Ein lokaler LLM-basierter Agent für barrierearme Webseitennutzung

vorgelegt am 05. August 2026

Name:	Theo Leuthardt
Matrikelnummer:	77205844868
Ausbildungsbetrieb:	Bundesdruckerei GmbH
Studienjahrgang:	2023
Fachbereich:	Duales Studium Wirtschaft · Technik
Studiengang:	Informatik
Betreuer/in Hochschule:	(Prof. Dr. rer. nat.) Björn Grohmann
Betreuer/in Unternehmen:	Diego Barrera Martinez
Anzahl der Wörter:	9892

Theo Leuthardt

Abstract

Durch fortschreitende Digitalisierung wird Barrierefreiheit in digitalen Medien wachsende Bedeutung zugeordnet, da immer mehr der alltäglichen Handlungen über das World Wide Web geschehen. Nach Schätzungen der Weltgesundheitsorganisation (WHO) wird die Zahl der Menschen mit einer erheblichen Behinderung weltweit auf 1,3 Milliarden beziffert [World Health Organization, 2022, S.23]. Trotz der Verbindlichkeit von Standards wie den Web Content Accessibility Guidelines (WCAG) wird in einer WebAIM-Analyse von 2024 festgestellt, dass 95,9% der meistbesuchten Webseiten gegen deren Anforderungen verstoßen [WebAIM, 2024b]. Beeinträchtigt durch diese Diskrepanz werden Menschen mit einer motorischen oder Sehbehinderung, denen dadurch die Bedienung von Webseiten, neben der Hürde der Maus- und Tastatur-Bedienung, erschwert wird. Daraus wird die Forschungsfrage dieser Arbeit abgeleitet, wie ein technisches Hilfsmittel entworfen werden kann, durch das Webseiten von dieser Zielgruppe ausschließlich per Spracheingabe bedient werden können, unter der Bedingung, dass sämtliche eingesetzte Künstliche Intelligenz (KI)-Komponenten vollständig lokal auf dem Endgerät ausgeführt werden.

Anhand einer Analyse des aktuellen Forschungs- und Softwarestands zu sprachgesteuerten und Large Language Model (LLM)-basierten Web-Agenten wird gezeigt, dass bislang keine Lösung eine barrierearme Sprachsteuerung mit vollständig lokaler KI-Ausführung im Browser inklusive geringem Installationsaufwand vereint. Auf dieser Erkenntnis aufbauend wird eine Browser-Erweiterung entworfen und implementiert, in der gesprochene Nutzerbefehle lokal transkribiert und durch ein ebenfalls lokal ausgeführtes Sprachmodell in konkrete Aktionen auf der jeweiligen Webseite übersetzt werden.

Erprobt wird der entstandene Prototyp anhand eines Einzelfalltests mit einer sehbehinderten Testperson, der mehrere realitätsnahe Aufgaben für die ausschließliche Befehlseingabe per Sprache gestellt werden. Dafür wird sowohl eine qualitative Beobachtung samt Interview als auch Telemetriedaten der Tests herangezogen, womit sich einerseits das Potential und andererseits technische Grenzen der Lösung herausarbeiten lassen. Die dabei gewonnenen Erkenntnisse werden im Rahmen dieser Arbeit ausführlich diskutiert und die Machbarkeit einer komplett lokalen barrierearmen Sprachsteuerung von Webseiten eingeordnet. Mit dieser Arbeit wird demnach ein Beitrag zur Frage geleistet, wie die digitale Teilhabe von Menschen mit einer motorischen oder Sehbehinderung durch lokale KI-Technologie im Browser verbessert werden kann.

Inhaltsverzeichnis

Abstract	I
Inhaltsverzeichnis	III
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Codeverzeichnis	VI
Akronyme	VII
Glossar	IX
1 Einführung	1
1.1 Einleitung	1
1.2 Problemstellung	2
2 Grundlagen	3
2.1 Barrierefreiheit und assistive Technologien im Web	3
2.2 Spracherkennung und natürlichsprachliche Verarbeitung	4
2.3 Browser-Extensions	5
3 Analyse	7
3.1 Stand der Technik	7
3.1.1 Überblick aktueller Forschung zu LLM-basierten Web-Agenten	7
3.1.2 In-Browser Agents for Search Assistance	8
3.1.3 Open-Source Software	9
3.1.4 Kritische Einordnung & Lösungsansatz	10
3.2 Anforderungsanalyse	11
3.2.1 Funktionale Anforderungen	11
3.2.2 Nicht-funktionale Anforderungen	14
4 Entwurf	16
4.1 Systemarchitektur	16
4.2 Interaktionskonzept	18
4.3 Technologieauswahl	21
5 Implementierung	23
5.1 Aufbau der Browser-Extension	23
5.1.1 Orchestrierung über Message-Router und Zustandsautomat	24

5.1.2	Lazy-Initialisierung von Offscreen-Dokument und Berechtigungs-Tab	24
5.2	Audioaufnahme und Spracherkennung	25
5.2.1	Aufnahme im Offscreen-Dokument	25
5.2.2	Lokale Transkription	26
5.2.3	Konfidenzbewertung und Nutzer-Feedback	26
5.3	Aktionsplanung mit dem lokalen LLM	26
5.3.1	Modell-Laden und grammatikgestützte Generierung	26
5.3.2	Document Object Model (DOM)-Serialisierung und Kontextreduktion . . .	27
5.3.3	Prompt-Engineering und Aktionshinweise	28
5.3.4	Antwort-Verarbeitung und Validierung	28
5.3.5	Replan-Schleife mit Konvergenzsicherungen	30
5.3.6	Regressionsverlauf und deterministischer Fallback	31
5.4	DOM-Interaktion	32
5.5	Popup-User-Interface	34
5.5.1	Statusanzeige und Aufnahmesteuerung	34
5.5.2	Aktionsprotokoll und Erfolgsfeedback	35
5.5.3	Telemetrie-Export	35
5.5.4	Barrierefreie Gestaltung	36
5.6	Fehlerbehandlung	37
6	Evaluation und Diskussion	38
6.1	Testszenarien und Ergebnisse	38
6.1.1	Testaufbau	38
6.1.2	Aufgabenergebnisse	39
6.1.3	Telemetrie	40
6.1.4	Qualitatives Feedback	40
6.2	Zielerfüllung und Limitierungen	41
6.2.1	Erfüllte Anforderungen	41
6.2.2	Nicht bzw. teilweise erfüllte Anforderungen	41
6.2.3	Limitierungen	42
7	Fazit	43
	Literaturverzeichnis	44
	KI-Verzeichnis	49
	Anhang	50
7.1	Testprotokoll TP1	50
	Ehrenwörtliche Erklärung	71

Abbildungsverzeichnis

1	Systemarchitektur-Übersicht	16
2	Interaktionskonzept: Aktivitätsdiagramm	19
3	Interaktionskonzept: Sequenzdiagramm	20

Tabellenverzeichnis

1	Funktionale Anforderungen: Sprachsteuerung	11
2	Funktionale Anforderungen: Natürlichsprachliche Verarbeitung & Aktionsplanung .	12
3	Funktionale Anforderungen: DOM-Interaktion	12
4	Funktionale Anforderungen: Popup-User-Interface	13
5	Funktionale Anforderungen: Fehlerbehandlung und Feedback	13
6	Nicht-funktionale Anforderungen: Lokale Ausführung und Datenschutz	14
7	Nicht-funktionale Anforderungen: Barrierefreiheit	14
8	Nicht-funktionale Anforderungen: Browser-Kompatibilität	15
9	Nicht-funktionale Anforderungen: Performance	15
10	Nicht-funktionale Anforderungen: Zuverlässigkeit	15
11	Testaufgaben mit gesprochenem Befehl	39
12	Verdichtete Aufgabenergebnisse	39
13	Subjektiv geschätzte vs. gemessene Wartezeit je Aufgabe	40
14	Abschlussinterview mit TP1	41

Codeverzeichnis

5.1	Manifest-Konfiguration der Extension	23
5.2	Generischer Message-Router aus <code>utils/messaging.ts</code>	24
5.3	Einmalige Berechtigungsanfrage in <code>entrypoints/permission/main.ts</code>	25
5.4	Aufnahmestart in <code>entrypoints/offscreen/recorder.ts</code>	25
5.5	Grammatikgestützte Generierung in <code>entrypoints/offscreen/action-planner/model.ts</code>	27
5.6	Element-Ranking in <code>entrypoints/offscreen/action-planner/prompt.ts</code>	27
5.7	Extraktion des JSON-Blocks in <code>entrypoints/offscreen/action-planner/response-parser.ts</code>	28
5.8	Sicheres JSON-Parsing in <code>entrypoints/offscreen/action-planner/response-parser.ts</code>	29
5.9	Validierung und Typ-Korrektur in <code>entrypoints/offscreen/action-planner/validation.ts</code>	29
5.10	Deterministisches Abschneiden in <code>action-planning-utils.ts</code>	30
5.11	Generation-Zähler in <code>entrypoints/background/state.ts</code>	30
5.12	Ausschnitt aus dem Replan-Entscheidungsknoten in <code>entrypoints/background/handlers/ action-planning.ts</code>	31
5.13	Deterministischer Fallback-Planer in <code>entrypoints/offscreen/action-planner/prompt.ts</code>	32
5.14	Selektor-Vergabe in <code>entrypoints/content/dom-extractor.ts</code>	33
5.15	Framework-kompatibler Wert-Setter in <code>entrypoints/content/action-executer.ts</code>	33
5.16	DOM-Beruhigung in <code>entrypoints/content/dom-settle.ts</code>	33
5.17	Nachrichtenempfang in <code>entrypoints/popup/hooks/usePipelineStatus.ts</code>	34
5.18	Statusrückmeldung nach Ausführung in <code>entrypoints/background/handlers/action-planning. ts</code>	35
5.19	Darstellung einer Aktion in <code>entrypoints/popup/components/ActionRow.tsx</code>	35
5.20	Farbvariablen für hohe Kontraste in <code>entrypoints/popup/App.css</code>	36

Akronyme

API Application Programming Interface

ARIA Accessible Rich Internet Applications

ASR Automatic Speech Recognition

CAPTCHA Completely Automated Public Turing test to tell Computers and Humans Apart

CPU Central Processing Unit

CSS Cascading Style Sheets

DOM Document Object Model

DSGVO Datenschutz-Grundverordnung

EU Europäische Union

GBNF GGML Backus-Naur Form

GGUF GPT-Generated Unified Format

GPT Generative Pre-trained Transformer

GPU Graphics Processing Unit

HTML Hypertext Markup Language

JSON JavaScript Object Notation

KI Künstliche Intelligenz

LLM Large Language Model

ML Machine Learning

NLP Natural Language Processing

ONNX Open Neural Network Exchange

PCM Pulse-Code Modulation

RAM Random Access Memory

UI User Interface

WCAG Web Content Accessibility Guidelines

WHO Weltgesundheitsorganisation

Glossar

Bildschirmlupe Assistives System, das einen Ausschnitt des Bildschirms stark vergrößert darstellt und Nutzern mit einer Sehbehinderung die Erkennung von Inhalten erleichtert. [World Health Organization, 2022, S.23]

Content Script Von einer Browser-Erweiterung mitgeliefertes Skript, das direkt in den Kontext einer Webseite eingebettet wird und somit Lese- und Schreibzugriff auf das DOM der Seite erhält. [Mozilla, 2024b]

Finite State Machine Endlicher Automat, der einen Verarbeitungsstatus über endliche Zustände und determinierte Übergänge führt und eingesetzt wird, um den Verarbeitungsstatus der Erweiterung bei Seitenwechseln konsistent zu halten. [Rabin and Scott, 1959, S.116]

Halluzination Von einem Sprachmodell erzeugte Ausgabe, die plausibel wirkt, aber sachlich falsch oder erfunden ist, im Kontext der Aktionsplanung etwa durch nicht existierende Selektoren auffallend. [Browser Use Team, 2024]

Inferenz Ausführung eines trainierten Modells zur Erzeugung einer Ausgabe aus einer Eingabe, im Gegensatz zum Training und bei lokalen Modellen direkt auf dem Endgerät des Nutzers durchgeführt. [Zheng et al., 2025, S.209]

Manifest V3 Aktuelle Version der Manifest-Spezifikation für Browser-Erweiterungen, die unter anderem Service Worker als Hintergrundskripte vorschreibt und gegenüber Manifest V2 verbesserte Sicherheit, Leistung und Privatsphäre sowie die browserübergreifende Lauffähigkeit unter Chrome, Firefox und Safari einführt. [Pantelaio and Kapravelos, 2024, S.2]

Offscreen Document Unter Manifest V3 einziger Ausführungskontext, der ein dauerhaftes, DOM-basiertes Ausführungsumfeld im Hintergrund bereitstellt, wodurch er für die lokale Ausführung von Modellinferenz genutzt wird, zu der ein Service Worker nicht fähig ist. [Google, 2024a]

Quantisierung Verfahren, das die Anzahl der pro Gewicht gespeicherten Bits reduziert, wodurch der Speicherbedarf und die Inferenzlatenz eines Modells auf handelsüblicher Hardware gesenkt werden. [Shen et al., 2024, S.783]

Screenreader Assistives System, das den Bildschirminhalt synthetisch spricht oder auf eine Braille-Zeile ausgibt und blinden oder sehbehinderten Nutzern die Bedienung digitaler Oberflächen ermöglicht. [WebAIM, 2024a]

Selektor Muster, über das ein Element im DOM ausgewählt wird, in CSS zum Stylen und in JavaScript zur Auswahl von DOM-Knoten, wobei HTML-Elemente anhand ihres Typs, ihrer Attribute, aktuellen Zustände und Position im DOM sowie über Kombinatoren anhand ihrer Beziehung zu anderen Elementen zielgenau angesprochen werden. [Mozilla, 2024d]

Service Worker In Browser-Erweiterungen unter Manifest V3 als Hintergrundskript eingesetzter Worker, der dauerhaft im Hintergrund läuft, den Zustand hält und die Kommunikation zwischen den Kontexten vermittelt, jedoch keinen direkten Zugriff auf das DOM einer Webseite besitzt. [Google, 2024d]

WebAssembly Binäres Instruktionsformat für eine stackbasierte virtuelle Maschine, das die Ausführung nativ kompilierter Bibliotheken wie llama.cpp plattformunabhängig innerhalb des Browsers ermöglicht. [Zhang et al., 2025, S.3]

1 Einführung

Im folgenden Kapitel wird der thematische Rahmen eingeführt. Zur Erarbeitung der Forschungsfrage wird in Abschnitt 1.1 der gesellschaftliche und technologische Kontext umrissen. Daran anschließend wird in Abschnitt 1.2 das Problem präzisiert und konkrete Defizite erklärt. Abschließend wird daraus die Forschungsfrage abgeleitet.

1.1 Einleitung

In den vergangenen Jahrzehnten hat sich das World Wide Web zur zentralen Infrastruktur entwickelt für den Informationsaustausch, die Kommunikation und das Anbieten sowie Nutzen digitaler Dienstleistungen. Alltägliche Handlungen wie Banktransaktionen, Terminvereinbarungen in der Medizin oder ein Identitätsnachweis setzen zunehmend auf Webtechnologien. Mit einer wachsenden Zahl solcher digitalen Dienstleistungen im Web gewinnt die Bedienbarkeit an Bedeutung, denn diese Entwicklung inkludiert nicht alle Menschen gleichermaßen.

Die WHO schätzt die Anzahl der Menschen mit erheblicher Behinderung auf 1,3 Milliarden, also 16% der Weltbevölkerung im Jahr 2021 [World Health Organization, 2022, S.23]. Mit dieser Zahl wird verdeutlicht, dass keine kleine Zielgruppe betroffen ist, sondern für einen zentralen Bestandteil der Bevölkerung mit digitaler Partizipation. Zudem wird im WHO-Report vermerkt, dass wenn zusätzliche Umweltbarrieren berücksichtigt werden, die reale Zahl der betroffenen Personen noch höher vermutet wird. Werden also digitale Systeme unzugänglich gestaltet, entstehen weitere Benachteiligungen oder bestehende werden verstärkt [World Health Organization, 2022, S.23].

Wird im WHO-Report thematisiert, dass mit der Einhaltung von internationalen Standards wie den WCAG Webinhalte für Menschen mit Behinderung in unterschiedlichen Kontexten zugänglich gemacht werden können. Zusätzlich werden öffentliche Stellen durch die Europäische Union (EU)-Richtlinie seit 2021 rechtlich verpflichtet, diesen internationalen Standard einzuhalten [Europäisches Parlament und Rat der Europäischen Union, 2016a]. Trotz der Existenz dieses Standards und der rechtlichen Verpflichtung durch die EU wird mit einer Analyse von WebAIM gezeigt, dass im Jahr 2024 95,9% der meist aufgerufenen Webseiten gegen die Anforderungen der WCAG verstoßen [WebAIM, 2024b]. Der Anspruch von Barrierefreiheit wird damit als hinter den normativen Ansprüchen betrachtet. Diese Diskrepanz zwischen Anspruch und Realität wird in der vorliegenden Arbeit weiter vertieft.

1.2 Problemstellung

Assistive Technologien für Webseiten wie Screenreader werden von blinden und sehbehinderten Nutzern zur Navigation digitaler Oberflächen verwendet. Die Umfrage WebAIM Screen Reader Survey #10 nutzt 1539 gültige Antworten von Menschen mit Screen Readern und zeigt damit die Relevanz dieser Nutzergruppe für die Bewertung der Barrierefreiheit von Webseiten [WebAIM, 2024a]. Screen Reader Nutzer werden befragt nach den problematischsten Barrieren, die sie auf Webseiten erleben, darunter Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA)-Verifizierungen, nicht erwartungsgemäß funktionierende interaktive Elemente wie Menüs oder Dialoge, nicht sinngemäße Links oder Buttons, fehlende Tastaturzugänglichkeit und komplexe Formulare [WebAIM, 2024a]. Moderne Webseiten mit komplexen Layouts, dynamischen Inhalten und visuell geprägten Strukturen erschweren die Navigation, da Screenreader auf klare Hierarchien, sinnvolle Beschriftungen und konsistente Interaktionsmuster angewiesen sind [Yu et al., 2025, S.1].

Aber nicht nur sehbehinderte Nutzer werden durch Barrieren eingeschränkt. Menschen, die Erkrankungen haben wie Parkinson, Zerebralparese oder Dyspraxie und damit in ihrer Feinmotorik eingeschränkt sind, werden ebenfalls mit herkömmlichen Computerschnittstellen vor Herausforderungen gestellt. Auch wenn Webinhalte visuell wahrgenommen werden können, stellen Maus, Tastatur oder andere Interaktionsgeräte mit einem Computer eine eingeschränkte Nutzbarkeit dar [Cossovic et al., 2023, S.3]. Als Lösung wird der Ansatz gesehen, Sprachsteuerung über LLMs zu realisieren, wodurch Befehle in natürlicher Sprache zur Navigation komplexer Benutzeroberflächen interpretiert werden. Durch Forschungsarbeiten wird eine sprachgesteuerte Webnavigation realisiert, bei der gesprochene Befehle in Interaktionen mit Webseiten übersetzt werden, um Barrieren für Menschen mit Beeinträchtigungen abzubauen [Pucci et al., 2024, S.1]. Bei der Nutzung von LLMs mit natürlicher Sprachverarbeitung wird jedoch das Problem erzeugt, dass personenbezogene Daten verarbeitet werden und ein Schutz vor Datenschutzverletzungen und rechtliche Anforderungen wie die Datenschutz-Grundverordnung (DSGVO) erforderlich werden [Sousa and Kern, 2023, vgl. S.1427].

Zudem wird durch Sousa und Kern hervorgehoben, dass personenbezogene Daten bei natürlicher Sprachverarbeitung nicht das Gerät des Nutzers verlassen sollten [Sousa and Kern, 2023, vgl. S.1442].

Daraus ergibt sich die folgende Forschungsfrage dieser Arbeit: Wie kann ein technisches Hilfsmittel entworfen werden, sodass es Menschen mit einer Seh- oder motorischen Behinderung ermöglicht wird, Webseiten ausschließlich durch Spracheingabe zu bedienen, unter der Bedingung, dass verwendete KI-Komponenten lokal auf dem Endgerät des Nutzers ausgeführt werden?

Im folgenden Kapitel werden die benötigten Grundlagen zur Barrierefreiheit, assistiven Technologien im Web, Spracherkennung, natürlichsprachlicher Verarbeitung und Browser-Extensions weiter vertieft.

2 Grundlagen

In diesem Kapitel werden die relevanten Grundlagen für die nachfolgenden Kapitel dargelegt. In Abschnitt 2.1 werden die Grundsätze der Barrierefreiheit und assistiver Technologien im Web erklärt und in Abschnitt 2.2 die für die Sprachsteuerung wichtigen Grundlagen. Zur Entwicklung des technischen Hilfsmittels werden die Grundstrukturen in Abschnitt 2.3 beschrieben.

2.1 Barrierefreiheit und assistive Technologien im Web

Barrierefreiheit im Web wird definiert als Gestaltung von Webinhalten, wodurch Menschen mit visuellen, auditorischen, motorischen, körperlichen, kognitiven, sprachlichen, Lern- und neurologischen Behinderungen auf Informationen und digitale Dienste zugreifen können. Als technischer Standard gelten die WCAG des World Wide Web Consortiums in der aktuellen Fassung 2.2, die seit Dezember 2024 als Empfehlung des W3C vorliegt. Gegliedert werden die Anforderungen an barrierefreie Webseiten in vier Grundsätze: Wahrnehmbarkeit, Bedienbarkeit, Verständlichkeit und Robustheit. Diesen Grundsätzen werden 13 Richtlinien zugeordnet, die grundlegende Ziele für testbare Erfolgskriterien festlegen. Jedes Erfolgskriterium wird in einer der drei Konformitätsstufen eingeordnet: A als niedrigste Mindestanforderung, AA und AAA als die höchsten Anforderungen [World Wide Web Consortium (W3C), 2023d]. Öffentliche Stellen werden in der Europäischen Union verpflichtet, die EU-Richtlinie 2016/2102 einzuhalten. Gemäß Artikel 1 wird festgelegt, dass die Anforderungen an die Barrierefreiheit aus Artikel 4 auf Webseiten und mobile Anwendungen einzuhalten sind [Europäisches Parlament und Rat der Europäischen Union, 2016a, S.8]. Dabei werden in Artikel 4 die Grundsätze der WCAG 2.2 präzisiert, wonach Webseiten und mobile Anwendungen wahrnehmbar, bedienbar, verständlich und robust gestaltet werden. Die WCAG 2.2 werden zwar nicht direkt genannt, jedoch werden die vier Grundsätze in das EU-Recht nahezu wortwörtlich überführt [Europäisches Parlament und Rat der Europäischen Union, 2016a, S.10].

Im WCAG 2.2 Kontext werden assistive Technologien als Hardware und/oder Software bezeichnet, die als Benutzeragent fungieren oder zusammen mit einem gängigen Benutzeragent verwendet werden [World Wide Web Consortium (W3C), 2023d]. Somit wird hier Hardware oder Software bezeichnet, durch die Funktionen für Nutzer mit Behinderungen ergänzend zu Standardbrowsern bereitgestellt werden. Zu den bereitgestellten Funktionen werden alternative Darstellungsformen wie vergrößerte Inhalte, alternative Eingabemethoden wie über eine Spracheingabe, zusätzliche Navigations- oder Orientierungsmechanismen und Inhaltsumwandlungen zum Beispiel bei Tabellen aufgezählt [World Wide Web Consortium (W3C), 2023d]. Screenreader werden im Bezugsrahmen der untersuchten assistiven Technologien für den Webzugang als zugehörig betrachtet.

Trotz der Existenz bestehender Standards und der teilweisen Verbindlichkeit durch die EU-Richtlinie 2016/2102 werden praktische Umsetzungen als nur unvollständig dokumentiert. Im WebAIM-Million-Report 2024 wird festgestellt, dass 95,9% der untersuchten Webseiten Fehler bezüglich WCAG 2 aufweisen. Als häufige Fehlerquellen werden mit 81% Anteil unzureichende Textkontraste, zu 54,5% fehlende Alternativtexte, fehlende Beschriftungen von Formulareingabefeldern zu 48,6%, leere Links mit 44,6% sowie leere Buttons zu 28,2% Anteil [WebAIM, 2024b]. Über diese Fehlertypen hinaus wird durch Yu et al. (2025) gezeigt, dass selbst auf Webseiten, die formalen Barrierefreiheitsvorgaben genügen, inkonsistente Überschriftenhierarchien und schlecht beschriftete interaktive Elemente die Navigation erheblich erschweren können [Yu et al., 2025, S.13]. Durch die WebAIM-Befragung wird die systemische Natur des Problems unterstrichen: Von 85,9% der befragten Screenreader-Nutzenden wird die Auffassung vertreten, dass zugänglichere Webseiten einen größeren Effekt auf die Web-Barrierefreiheit hätten als die Weiterentwicklung assistiver Technologien [WebAIM, 2024a].

2.2 Spracherkennung und natürlichsprachliche Verarbeitung

Spracherkennung wird im wissenschaftlichen Kontext als Automatic Speech Recognition (ASR) bezeichnet und umfasst die Umwandlung gesprochener Sprache in Text. Durch den Einsatz von Deep Learning wird die klassische Pipeline aus getrennten Teilkomponenten durch End-to-End-Modelle abgelöst, welche alle Schritte in einem neuronalen Netz gemeinsam trainieren, sodass ein einzelnes Modell ein Audiosignal unmittelbar in Text überführt [Prabhavalkar et al., 2023, S.1; S.16].

Unter LLMs werden hauptsächlich Transformer-basierte neuronale Sprachmodelle verstanden, die Dutzende bis Hunderte von Milliarden Parametern enthalten und anhand umfangreicher Textdaten vortrainiert werden. LLMs werden im Vergleich zu PLMs als wesentlich größer mit ausgeprägteren Fähigkeiten im Bereich des Sprachverständnisses und der Sprachgenerierung definiert. Vor allem emergente Fähigkeiten wie kontextbasiertes Lernen, das Befolgen von Anweisungen und mehrstufiges Schlussfolgern werden im Vergleich geschildert [Zhao et al., 2024, S.2]. Nach Wei et al. (2022) wird die Befolgung von Anweisungen als zunehmend wichtiger Forschungszweig gesehen, indem Sprachmodelle besser in die Lage versetzt werden, neue Aufgaben durch das Lesen von Anweisungen zu bewältigen. Durch das Fine-Tuning mit gemischten Aufgaben formuliert als Anweisungen wird erreicht angemessene Reaktionen auf Anweisungen von Sprachmodellen zu erhalten, die bisher unbekannte Aufgaben für das Sprachmodell beschreiben [Wei et al., 2022, S.5]. Dies wird außerhalb der Natural Language Processing (NLP)-Forschungsgemeinschaft zur Übersetzung von Anweisungen in natürlicher Sprache zu z.B. für Roboter ausführbaren Aktionen oder Nutzerinteraktion angewendet [Wei et al., 2022, S.10]. Aktionsplanung wird somit als Anwendungsmöglichkeit dargelegt und im Folgenden weiter vertieft.

Die Planung von Aktionen wird als wichtigste Fähigkeit von autonomen Agenten gesehen und erfordert komplexe Verständnis-, Schlussfolgerungs- und Entscheidungsprozesse. Allgemein wird trotz des abstrakten Konzepts von Aktionsplanung diese als Generierung einer Abfolge von Aktionen auf Basis der Umgebung, des Aktionsraums und des Aufgabenziels definiert. Eine Aktion wird als Element des Aktionsraums zu einem bestimmten Zeitschritt beschrieben und dient der zeitlichen Abfolge mehrerer Aktionen [Huang et al., 2024, S.1].

Zur Ausführung und Bereitstellung von Machine Learning (ML)-Modellen in Webanwendungen kann Open Neural Network Exchange (ONNX) Runtime Web mithilfe von JavaScript-Application Programming Interfaces (APIs) und -Bibliotheken verwendet werden. Je nach Entwicklungsumgebung wird durch ONNX Runtime Web ermöglicht, ML neben der Integration in frontendseitige Komponenten von Webanwendungen auch in serverseitige Komponenten durch Bibliotheken in anderen Programmiersprachen zu integrieren. Für die Inferenz im Browser wird die Verwendung des Paketes „onnxruntime-web“ angeboten und empfohlen. Dies erlaubt, WebGL, WebGPU oder WebNN für die Inferenz über Graphics Processing Unit (GPU)-Verarbeitung und WebAssembly oder WebNN für die Central Processing Unit (CPU)-Verarbeitung zu nutzen. Wird das ONNX Runtime Web Paket verwendet, werden Modelle im ONNX-Format benötigt, die in einer Kollektion auf GitHub vorliegen und eine Konvertierung bedingen [Microsoft, 2024]. Für LLMs im GPT-Generated Unified Format (GGUF)-Format wird durch das Paket `wllama` eine WebAssembly-Anbindung an `llama.cpp` bereitgestellt, wodurch Modelle in diesem Format ebenfalls im Browser ausgeführt werden können und aus lokalen URLs oder Modell-Repositories geladen werden [ngxson, 2024]. Das GGUF-Format wird als ein binäres Modellformat für die Inferenz mit GGML-basierten Laufzeiten definiert und wird von HuggingFace als Plattform zum Download von Modellen als unterstütztes Format bezeichnet [ggml-org, 2024]. Dadurch wird mit dem GGUF-Format gegenüber dem ONNX-Format eine größere Auswahl von Modellen durch die Plattform HuggingFace geboten.

2.3 Browser-Extensions

Browser werden um zusätzliche Funktionen und Features mithilfe von Browser-Extensions erweitert, die ebenfalls wie Webseiten mit den bekannten webbasierten Technologien Hypertext Markup Language (HTML), Cascading Style Sheets (CSS) und JavaScript entwickelt werden können. Zusätzlich zur möglichen Nutzung von Web-APIs wie bei einer Webseite wird ein Zugriff auf weitere JavaScript-APIs, WebExtensions-APIs genannt, gewährt. Durch Browser-Extensions wird es Nutzern durch den Zugriff und die Aktualisierung des HTML- und CSS-Codes ermöglicht, Inhalte von Webseiten zu entfernen oder neue hinzuzufügen, wie z.B. Werbeflexer. So werden neue Browser-Funktionen entwickelt und bestehende Webseiten in ihrer Nutzbarkeit verbessert [Mozilla, 2024j].

Eine Browser-Extension wird aus mehreren Komponenten zusammengesetzt. In der Datei `manifest.json` werden grundlegende Metadaten wie Name, Version und benötigte Berechtigungen der Extension festgelegt und als einzige notwendige Datei einer Browser-Extension deklariert. Content Scripts werden als mitgelieferter JavaScript-Code verwendet, der in Webseiten eingefügt wird, womit Inhalte von Webseiten gelesen und verändert werden können. Um unabhängig von der Lebensdauer einer bestimmten Webseite oder eines Browser-Fensters auf Browser-Ereignisse zu reagieren, werden Background Scripts eingesetzt. Background Scripts werden entweder persistent geladen und bleiben aktiv bis zur Deaktivierung der Browser-Extension oder werden nicht persistent nur bei Bedarf geladen und bei Inaktivität wieder entladen. Ebenfalls werden in Browser-Extensions HTML-Dokumente in Form von Sidebars, Popups oder Optionsseiten zur Entwicklung einer Benutzeroberfläche für eine Extension bereitgestellt. Die Kommunikation zwischen diesen Komponenten wird über Nachrichten realisiert, wodurch beispielsweise Content Scripts mit Background Scripts kommunizieren können, falls eine Komponente nicht die notwendigen Berechtigungen besitzt [Mozilla, 2024b].

Im folgenden Kapitel 3 wird der aktuelle Stand der Technik analysiert und darauf aufbauend die funktionalen und nicht-funktionalen Anforderungen definiert.

3 Analyse

In den nachfolgenden Abschnitten wird der Stand der Technik analysiert und darauf basierend die Anforderungen zur Beantwortung der Forschungsfrage definiert. In Abschnitt 3.1 werden vorhandene Forschung und Softwarelösungen für LLM-basierte Web-Agenten und deren Probleme analysiert. Darauf aufbauend werden in Abschnitt 3.2 die Anforderungen definiert, gegliedert in funktionale in Abschnitt 3.2.1 und nicht-funktionale in Abschnitt 3.2.2.

3.1 Stand der Technik

Zur Einordnung werden in den folgenden Sektionen Technologien und Forschungsarbeiten analysiert, die sich mit LLM-basierten Web-Agenten auseinandersetzen. In Abschnitt 3.1.1 wird ein Überblick zur aktuellen Forschung mit Analyse der Forschungsfragen geboten. Zusätzlich wird die Arbeit zu In-Browser Agents for Search Assistance in Abschnitt 3.1.2 untersucht. Vorhandene Open-Source-Software wird in Abschnitt 3.1.3 ausgewertet. Abschließend werden in Abschnitt 3.1.4 die Defizite aller analysierten Forschungen und Lösungen zusammengeführt und ein Lösungsansatz definiert.

3.1.1 Überblick aktueller Forschung zu LLM-basierten Web-Agenten

Durch Deng et al. (2023) wird folgende Forschungsfrage definiert, „Wie können wir einen universellen Agenten für das Internet entwickeln, der bei jeder beliebigen Website sprachliche Anweisungen befolgen und die entsprechenden Aufgaben ausführen kann?“ [Deng et al., 2023, S.1]. In dieser Forschungsarbeit wird zu Mind2Web ein Datensatz mit über 2000 offen formulierten Aufgaben aus 137 realen Webseiten und 31 Domänen als Grundlage universeller Web-Agenten geliefert [Deng et al., 2023, S.1]. Darüber hinaus wird mit MindAct ein zweistufiges Agentenmodell vorgestellt, das ein feinabgestimmtes kleines Sprachmodell zur Filterung von Web-Elementen und ein LLM zur Aktionsplanung umfasst [Deng et al., 2023, S.1]. Zugänglichkeit für Menschen mit Behinderung wird als positive Chance bewertet [Deng et al., 2023, S.9]. Als Limitationen wird aufgezählt, dass MindAct Webseiten nur textbasiert verarbeitet und jede Seite isoliert modelliert wird, wodurch dynamische Zustandsänderungen nach einer Nutzerinteraktion nicht berücksichtigt werden [Deng et al., 2023, S.10]. Ebenfalls wird bemängelt, dass während einer Aufgabe keine Korrekturen vom Nutzer möglich sind und die Evaluation auf gecachten Offline-Snapshots False Negatives trotz alternativer gültiger Aktionspfade ermöglicht [Deng et al., 2023, S.10].

Der Ansatz von Mind2Web wird durch WebVoyager um visuelle Wahrnehmung erweitert, indem aus Screenshots und HTML-Text eine menschenähnliche Wahrnehmung erzeugt und direkt auf der echten Webseite agiert wird [He et al., 2024, S.1]. Als Einschränkung wird aufgezählt, dass das visuelle System durch textlastige Webseiten überfordert wird und Login-Verfahren sowie CAPTCHA vermieden werden müssen [He et al., 2024, S.4] [He et al., 2024, S.8].

Mit SeeAct wird von Zheng et al. (2024) Generative Pre-trained Transformer (GPT)-4V als universeller Agent untersucht, wobei Planung und Ausführung getrennt und ein generierter Textplan durch ein separates Grounding-Modul in ein konkretes HTML-Element übersetzt wird [Zheng et al., 2024, S.2]. Als Befund wird festgehalten, dass visuelles Grounding das Kernproblem bleibt, mit einer Lücke von 20 bis 25 Prozentpunkten zum theoretischen Optimum [Zheng et al., 2024, S.8]. Mit AutoWebGLM wird mit der Cloud-Abhängigkeit gebrochen und gezeigt, dass ein Open-Source-Modell mit nur sechs Milliarden Parametern (ChatGLM3-6B) durch HTML-Vereinfachung, Curriculum Learning und Reinforcement Learning von proprietären Modellen auf Benchmarks übertroffen werden kann [Lai et al., 2024, S.1]. Dabei wird als Limitation ein fehlender universeller Aktionsraum benannt und festgestellt, dass der Agent nicht aus fehlerhaften Iterationsschleifen befreit werden kann [Lai et al., 2024, S.10].

Durch WebRL wird belegt, dass die Leistungslücke zwischen Open-Source und proprietären LLMs auf fehlende entscheidungszentrierte Trainingsdaten zurückzuführen ist, und mit einem Reinforcement-Learning-Framework geschlossen wird, indem neue Trainingstasks aus fehlgeschlagenen Versuchen generiert werden [Qi et al., 2024, S.2]. Dabei wird Llama-3.1-8B von 4,8% auf 42,4% auf WebArena-Lite gesteigert und GPT-4-Turbo (17,6%) übertroffen, jedoch wird die Abhängigkeit zu aufwendiger Trainingsinfrastruktur aufrechterhalten [Qi et al., 2024, S.1].

Von WebChallenger wird argumentiert, dass das Problem nicht in der Modellgröße gesehen wird, sondern in Agentenarchitekturen, in denen drei menschliche kognitive Vorteile nicht repliziert werden, nämlich selektive Aufmerksamkeit, persistentes Gedächtnis und prozedurale Handlungsrouinen, die durch eine strukturierte DOM-Repräsentation namens PageMem adressiert werden [Hwang et al., 2026, S.1]. Geprüft wird mit der Forschungsfrage, ob Open-Weight-Modelle ohne Fine-Tuning durch eine bessere Architektur auf Niveau proprietärer Systeme gebracht werden können, was mit 56,3% auf WebArena bestätigt wird [Hwang et al., 2026, S.7]. Benannt wird als Limitation, dass die Laufzeit pro Aufgabe durch eine große Zahl sequenzieller LLM-Aufrufe erhöht wird und das Framework mit Frontier-Modellen als teuer im Betrieb eingestuft wird [Hwang et al., 2026, S.27].

3.1.2 In-Browser Agents for Search Assistance

Durch Zerhoudi und Granitzer (2026) wird ein fundamentaler Konflikt zwischen dem Bedarf an KI-Assistenz und dem Schutz privater Browserdaten adressiert, da Nutzer durch aktuelle Modelle gezwungen werden, sensible Daten an externe Dienste zu senden [Zerhoudi and Granitzer, 2026, S.345]. Als Ziel wird eine voll funktionsfähige Alternative innerhalb des Browsers formuliert, die ohne Cloud-Anbindung betrieben werden kann [Zerhoudi and Granitzer, 2026, S.346].

Als Systemarchitektur wird ein hybrider Ansatz gewählt, in dem die Suchstrategie des Nutzers mittels direktem Feedback im Online-Learning durch ein lokales probabilistisches Modell gelernt wird und dieses als Steuerungsmechanismus für ein Small Language Model eingesetzt wird, durch das Handlungsvorschläge generiert und der Suchraum für die KI eingeschränkt werden. Für die lokale Implementierung wird die gesamte Berechnung über WebWorker und WebLLM bzw. WebGPU für die Inferenz ausgeführt, wobei Modelle der Phi-Familie mit ca. 2,7 Milliarden Parametern verwendet werden und alle Nutzerinteraktionen ausschließlich in der IndexedDB des Browsers gespeichert werden [Zerhoubi and Granitzer, 2026, S.346] [Zerhoubi and Granitzer, 2026, S.347]. In einer dreiwöchigen Nutzerstudie mit 18 Probanden wird die durchschnittliche Session-Länge signifikant von 6,8 auf 5,2 Suchanfragen gesenkt und für die Vorschläge der KI eine Akzeptanzrate von 36,4% gemessen [Zerhoubi and Granitzer, 2026, S.348]. Als Limitation wird benannt, dass sich die Evaluation primär auf eine universitäre Stichprobe stützt und das Modell vor allem mit der eigenen generischen Baseline verglichen wird, nicht mit komplexeren externen Empfehlungssystemen [Zerhoubi and Granitzer, 2026, S.348].

3.1.3 Open-Source Software

Mit Browser-Use wird ein Open-Source-Framework in Python bereitgestellt, durch das LLMs befähigt werden, Webbrowser wie menschliche Nutzer zu bedienen, indem das DOM in ein für KI lesbares Format transformiert wird [Browser Use Team, 2024]. Als Architektur wird auf einem Agenten als Orchestrator, einem Controller für die Aktionsausführung und einem BrowserSession-Layer (Playwright) aufgebaut, in dem ein DOM-Service zur Identifikation interaktiver Elemente genutzt wird [Browser Use Team, 2024]. Eingesetzt wird das Framework für komplexe Web-Automatisierungen, wobei der Fokus auf die Flexibilität gegenüber Layout-Änderungen gelegt wird, da der Kontext der Seite durch das Modell verstanden wird, statt dass nur statische Selektoren verwendet werden [Browser Use Team, 2024]. Als Limitationen werden hohe Token-Kosten bei komplexen Seiten und Latenzzeiten durch die Modell-Interaktion benannt, zudem wird das Risiko von Halluzinationen bei der Aktionsplanung festgestellt [Browser Use Team, 2024].

Durch Firefox Voice wird eine Browser-Erweiterung für Firefox als offene Alternative zu kommerziellen Assistenten wie Alexa oder Google Assistant bereitgestellt, mit der Sprachsteuerung direkt im Web-Kontext ermöglicht wird [Cambre et al., 2021, S.1]. Auf Architekturebene wird eine Cloud-basierte Spracherkennung genutzt, jedoch wird die Verarbeitung der Absicht lokal in der Erweiterung vorgenommen, damit die Kontrolle über die Daten und Interaktionsmöglichkeiten beibehalten wird [Cambre et al., 2021, S.2]. In der öffentlichen Bereitstellung werden über 12.000 aktive Nutzer erreicht, und aufgrund der leichten Erweiterbarkeit wird Firefox Voice als quelloffene Grundlage bereitgestellt, durch die von Entwicklern und Forschern mit Sprachinteraktion experimentiert werden kann [Cambre et al., 2021, S.2]. Als Limitation wird festgestellt, dass im Gegensatz zu Browser-Use nicht auf LLMs gesetzt wird, sondern auf vordefinierten Grammatiken und Intents, wodurch die Flexibilität bei unbekannten Webseiten eingeschränkt wird [Cambre et al., 2021, S.5].

3.1.4 Kritische Einordnung & Lösungsansatz

Zusammenfassend wird über die betrachteten Arbeiten hinweg eine durchgehende Forschungslücke sichtbar, da der Fokus bestehender Ansätze entweder auf cloudbasierte proprietäre Modelle mit separatem Grounding-Modul [Zheng et al., 2024, S.8], auf Trainingsinfrastrukturen für Open-Source-LLMs [Qi et al., 2024, S.1] oder auf Architekturverbesserungen statt Modellskalierung [Hwang et al., 2026, S.1] gelegt wird. Zudem wird als wiederkehrendes Problem abgeleitet, dass dynamische Zustandsänderungen nach Nutzerinteraktionen nicht berücksichtigt [Deng et al., 2023, S.10] und fehlerhafte Iterationsschleifen nicht robust behandelt werden [Lai et al., 2024, S.10]. Wie aus Abschnitt 3.1 hervorgeht, wird Barrierefreiheit in keiner der untersuchten Forschungsarbeiten als primäres Designziel gesetzt und eine echte Sprachsteuerung gar nicht adressiert. Aus der Untersuchung der In-Browser Agents for Search Assistance wird erkennbar, dass eine lokale Inferenz im Browser technisch umsetzbar ist, jedoch wird nur ein Szenario für die Suchassistenz abgedeckt und eine LLM-basierte Aktionsplanung nicht vorgesehen [Zerhoubi and Granitzer, 2026, S.347]. Aus der Analyse der Open-Source-Software wird ersichtlich, dass Browser-Use nicht im Browser eingebettet ist, während Firefox Voice eine Sprachsteuerung im Web-Kontext bereitstellt, aber auf Cloud-basierte Spracherkennung zurückgreift [Browser Use Team, 2024] [Cambre et al., 2021, S.2].

Da in keiner der analysierten Arbeiten lokale kleine KI-Modelle, direkte In-Browser-Integration und Spracheingabe für barrierearme Webseitennutzung kombiniert werden, wird eine Browser-Erweiterung mit niedrigem Installationsaufwand als Marktlücke identifiziert, die vollständig auf lokaler KI basiert und durch schlanke Modelle barrierearme Sprachsteuerung auf Endgeräten ohne spezialisierte Hardware ermöglicht.

Als Realisierungsform wird eine Browser-Erweiterung gegenüber alternativen Ansätzen abgegrenzt, weil ein cloudbasierter Webservice durch die Bedingung der lokalen Ausführung ausgeschlossen wird, eine eigenständige Anwendung wie bei Browser-Use auf einen externen Treiber angewiesen wäre und von einem Userscript weder persistente Hintergrund-Prozesse noch eine eigene Oberfläche bereitgestellt werden [Browser Use Team, 2024]. Bei einem niedrigen Installationsaufwand wird durch eine Browser-Erweiterung der direkte Zugriff auf das DOM im aktiven Tab über Content Scripts, eine persistente Steuerung über Background Scripts und eine zugängliche Oberfläche über ein Popup ermöglicht, wie in Abschnitt 2.3 dargelegt, wobei eine lokale Inferenz im Browser wie in Abschnitt 2.2 als umsetzbar eingestuft wird [Mozilla, 2024b] [Zerhoubi and Granitzer, 2026, S.346]. Diese Kombination wird in dieser Arbeit als Abgrenzung verfolgt, mit der Limitation, dass kein eigenes Modelltraining durchgeführt wird und die Evaluation auf lokaler Hardware vorgenommen wird. Daran angepasst wird die Forschungsfrage aus Abschnitt 1.1 folgend neu formuliert: Wie kann eine Browser-Erweiterung mit niedrigem Installationsaufwand entworfen werden, die Menschen mit einer motorischen oder Sehbehinderung ermöglicht, Webseiten ausschließlich durch Spracheingabe zu bedienen, unter der Bedingung, dass alle KI-Komponenten vollständig lokal im Browser inferiert werden? Aus dieser Frage und der vorhergehenden Analyse werden in Abschnitt 3.2 die funktionalen und nicht-funktionalen Anforderungen abgeleitet, die als Grundlage für die Implementierung der Browser-Erweiterung dienen.

3.2 Anforderungsanalyse

Wie in Abschnitt 3.1 herausgearbeitet, werden Barrierefreiheit und Sprachsteuerung in keiner untersuchten Lösung als primäres Designziel verfolgt. Da blinde und sehbehinderte Nutzer auf das Internet angewiesen sind, jedoch durch unzugängliche Webseitengestaltung eingeschränkt werden, kann durch technische Maßnahmen eine Verbesserung der Nutzbarkeit erreicht werden [Yu et al., 2025, S.2, S.3]. Dementsprechend wird die Zielgruppe auf Menschen mit motorischer oder Sehbehinderung festgelegt, für die Webseiten nur mit assistiven Technologien bedient werden können. Aus der Analyse und der Forschungsfrage werden in den folgenden Sektionen die erforderlichen Anforderungen zur Schließung dieser Lücke definiert. Unterteilt werden die Anforderungen in funktionale in Abschnitt 3.2.1 und nicht-funktionale in Abschnitt 3.2.2. Alle Anforderungen werden in Tabellen mit den Spalten ID, Beschreibung und Priorität dargestellt und in Themengebiete unterteilt.

3.2.1 Funktionale Anforderungen

Mit den funktionalen Anforderungen werden die Funktionalitäten der Software zur Sprachsteuerung von Webseiten beschrieben. Diese werden in die Teilbereiche Sprachsteuerung, natürlichsprachliche Verarbeitung & Aktionsplanung, DOM-Interaktion, Popup-User-Interface und Fehlerbehandlung & Feedback im Folgenden gegliedert.

Sprachsteuerung

Weil keine der bestehenden Lösungen aus Abschnitt 3.1 eine vollständige Spracherkennung mit Transkription enthält, wird in Tabelle 1 festgelegt, dass Audioerfassung und Transkription ohne Cloud-Anbindung erfolgen.

ID	Beschreibung	Priorität
FA-01	Es muss Audioaufnahme über das Mikrofon des Nutzers unterstützt werden, nach expliziter Berechtigungserteilung.	Muss
FA-02	Die gesprochene Eingabe muss lokal in Text transkribiert werden, ohne Daten an externe Server zu senden.	Muss
FA-03	Das Transkriptionssystem muss deutsche Spracheingaben verstehen und korrekt transkribieren.	Muss
FA-04	Die Audioaufnahme muss durch eine explizite Nutzeraktion gestartet und gestoppt werden können.	Muss
FA-05	Während der Audioaufnahme sollte dem Nutzer ein visuelles und/oder akustisches Feedback gegeben werden, dass die Aufnahme läuft.	Soll

Tabelle 1: Funktionale Anforderungen: Sprachsteuerung

Natürlichsprachliche Verarbeitung & Aktionsplanung

Zur Analyse der Nutzerintention wird im Gegensatz zu cloudbasierten Agenten in Tabelle 2 festgelegt, dass die Aktionsplanung auf Basis des Transkripts lokal im Browser erfolgt und ein deterministischer Aktionsplan ausgegeben wird.

ID	Beschreibung	Priorität
FA-06	Transkribierter Text muss an ein lokales LLM übergeben werden, das eine semantische Analyse durchführt.	Muss
FA-07	Das LLM muss basierend auf der Nutzerintention und dem aktuellen DOM einen strukturierten Aktionsplan generieren.	Muss
FA-08	Der Aktionsplan muss mindestens folgende Aktionstypen unterstützen: Klicken, Text eingeben, Option aus Dropdown auswählen, Formular absenden.	Muss
FA-09	Die DOM der aktuell geöffneten Webseite muss dem LLM zu Verfügung gestellt werden können.	Muss
FA-10	Das LLM muss mit unvollständigen oder mehrdeutigen Nutzereingaben umgehen können und ggf. einen alternativen Plan ableiten.	Soll

Tabelle 2: Funktionale Anforderungen: Natürlichsprachliche Verarbeitung & Aktionsplanung

DOM-Interaktion

Die DOM-Steuerung wird durch bestehende Lösungen entweder über externe Treiber wie Playwright bei Browser-Use aus Abschnitt 3.1.3, über ein separates Grounding-Modul wie bei SeeAct oder über modellseitige Elementselektion auf gecachten Snapshots wie bei Mind2Web aus Abschnitt 3.1.1 realisiert, wodurch ein Eingriff im aktiven Browser-Tab des Nutzers nicht vorgesehen wird [Browser Use Team, 2024] [Zheng et al., 2024, S.2] [Deng et al., 2023, S.10]. Daher wird in Tabelle 3 festgelegt, dass die Interaktion direkt im Browser-Tab über semantische Selektoren erfolgt.

ID	Beschreibung	Priorität
FA-11	Die Software muss den generierten Aktionsplan parsen und die enthaltenen Aktionen sequentiell im Browser ausführen.	Muss
FA-12	Klick-Aktionen müssen auf dem korrekten DOM-Element ausgeführt werden, identifiziert über semantische Merkmale.	Muss
FA-13	Texteingaben müssen in Formularfelder eingefügt werden können.	Muss
FA-14	Scroll-Aktionen können ausgeführt werden, um nicht sichtbare Elemente in den Viewport zu bringen.	Kann
FA-15	Nach jeder ausgeführten Aktion muss die übergebene DOM aktualisiert und dem LLM erneut übergeben werden können.	Soll

Tabelle 3: Funktionale Anforderungen: DOM-Interaktion

Popup-User-Interface

Wie in der kritischen Einordnung in Abschnitt 3.1.4 herausgearbeitet, wird in den untersuchten Lösungen Barrierefreiheit nicht als primäres Designziel vorgesehen, weshalb in Tabelle 4 ein zugängliches User-Interface festgelegt wird, über das die Sprachsteuerung bedient und der Ausführungsstatus nachvollzogen werden kann.

ID	Beschreibung	Priorität
FA-16	Es muss ein User-Interface bereitgestellt werden, über das die Aufnahme gestartet und gestoppt sowie der Status angezeigt werden kann.	Muss
FA-17	Das User-Interface muss den aktuellen Status der Aufgabenausführung anzeigen.	Muss
FA-18	Das User-Interface muss eine Liste der zuletzt ausgeführten Aktionen oder ein Protokoll der Interaktion anzeigen.	Kann

Tabelle 4: Funktionale Anforderungen: Popup-User-Interface

Fehlerbehandlung & Feedback

Durch Audio-Transkription und Aktionsplanung durch das LLM wird ein intransparentes Verhalten ermöglicht. Wie bei Browser-Use aus Abschnitt 3.1.3 wird dabei ein Risiko von Halluzinationen bei der Aktionsplanung festgestellt [Browser Use Team, 2024]. Bei AutoWebGLM aus Abschnitt 3.1.1 werden zudem fehlerhafte Iterationsschleifen nicht robust behandelt, wodurch die Zugänglichkeit gemindert wird [Lai et al., 2024, S.10]. In Tabelle 5 werden daher funktionale Anforderungen an Fehlerbehandlung und Feedback bestimmt.

ID	Beschreibung	Priorität
FA-19	Bei Fehlern in der Audioverarbeitung muss eine verständliche Fehlermeldung ausgegeben werden.	Muss
FA-20	Wenn das LLM keinen verständlichen Aktionsplan generieren kann, muss der Nutzer darüber informiert werden über eine Fehlermeldung.	Muss
FA-21	Wenn eine Aktion im Browser fehlschlägt, muss ein Fehlerstatus angezeigt und ggf. eine alternative Strategie versucht werden.	Soll
FA-22	Dem Nutzer muss nach erfolgreicher Ausführung eines Aktionsplans ein visuelles oder akustisches Erfolgsfeedback gegeben werden.	Soll

Tabelle 5: Funktionale Anforderungen: Fehlerbehandlung und Feedback

3.2.2 Nicht-funktionale Anforderungen

In diesem Abschnitt werden die nicht-funktionalen Anforderungen als Qualitätskriterien definiert.

Lokale Ausführung

Datenschutzrechtlich werden Verantwortliche durch die Datenschutz-Grundverordnung zu Datenminimierung und Zweckbindung verpflichtet, wodurch die Übertragung sensibler Webinhalte an cloudbasierte Server als riskant einzustufen ist [Europäisches Parlament und Rat der Europäischen Union, 2016b]. Häufig wird durch cloudbasierte Sprachassistenten eine externe Datenübertragung vorausgesetzt, wodurch Nutzer gezwungen werden, sensible Daten an externe Dienste zu senden [Zerhoubi and Granitzer, 2026, S.345]. Infolgedessen wird festgelegt, dass die Verarbeitung auf dem Endgerät erfolgt.

ID	Anforderung	Priorität
NFA-1	Alle Daten, also Sprach-, DOM- und Nutzerdaten, müssen vollständig lokal verarbeitet werden.	Muss
NFA-2	Nach initialem LLM-Download müssen alle Funktionalitäten ohne Internetverbindung erfolgen können.	Muss

Tabelle 6: Nicht-funktionale Anforderungen: Lokale Ausführung und Datenschutz

Barrierefreiheit

Da Barrierefreiheit als Ziel der Forschungsfrage vorgesehen und in bestehenden Lösungen vernachlässigt wird, wird in Tabelle 7 die Konformität mit WCAG 2.2 Level AA und Kompatibilität mit assistiven Technologien festgelegt.

ID	Anforderung	Priorität
NFA-3	Die Farbkontraste des User-Interface müssen mindestens WCAG AA erfüllen.	Muss
NFA-4	Alle Bedienelemente im User-Interface müssen über Tastatur erreichbar und bedienbar sein.	Muss
NFA-5	Das User-Interface muss minimalistisch und unaufdringlich gestaltet sein.	Soll
NFA-6	Kompatibilität mit ergänzenden assistiven Technologien wie Screenreader, Switch Access oder virtueller Tastatur muss gegeben sein.	Muss

Tabelle 7: Nicht-funktionale Anforderungen: Barrierefreiheit

Browser-Kompatibilität

Zur Gewährleistung von Plattformunabhängigkeit wird in Tabelle 8 die Lauffähigkeit in mehreren Browsern festgelegt.

ID	Anforderung	Priorität
NFA-7	Lauffähigkeit unter Google Chrome.	Muss
NFA-8	Lauffähigkeit unter Mozilla Firefox.	Kann
NFA-9	Lauffähigkeit unter Apple Safari.	Kann

Tabelle 8: Nicht-funktionale Anforderungen: Browser-Kompatibilität

Performance

Rechenintensive Modelle und wiederholte Downloads werden als akzeptanzmindernd betrachtet, weshalb in Tabelle 9 quantisierte gecachte Modelle mit akzeptabler Latenz auf aktueller Hardware gefordert werden.

ID	Anforderung	Priorität
NFA-10	Das erste Laden des LLMs sollte nicht länger als 30 Sekunden dauern.	Soll
NFA-11	Die Latenz zwischen Ende der Spracheingabe und Beginn der Ausführung sollte unter 10 Sekunden liegen.	Soll
NFA-12	Die Speichernutzung auf dem Endgerät sollte 2 GB RAM nicht überschreiten.	Soll
NFA-13	Das Transcript-Model sollte Echtzeit- oder Near-Echtzeit-Transkription unterstützen.	Soll
NFA-14	Modelle müssen im Browser-Cache persistiert werden, um wiederholte Downloads zu vermeiden.	Muss

Tabelle 9: Nicht-funktionale Anforderungen: Performance

Zuverlässigkeit

Zur Vermeidung unsteter Zustände und Konflikte mit anderen assistiven Technologien werden in Tabelle 10 Anforderungen zur Zuverlässigkeit bestimmt.

ID	Anforderung	Priorität
NFA-15	Beim Wechsel der Webseite oder Reload der Webseite muss der Zustand konsistent zurückgesetzt oder wiederhergestellt werden.	Muss
NFA-16	Es dürfen keine bestehenden assistiven Technologien behindert oder dessen Verhalten verändert werden.	Muss

Tabelle 10: Nicht-funktionale Anforderungen: Zuverlässigkeit

Auf Grundlage der Anforderungen wird im folgenden Kapitel 4 der Entwurf von System und Interaktion vorgenommen.

4 Entwurf

In den nachfolgenden Abschnitten wird der Entwurf der Browser-Extension auf Basis der in Abschnitt 3.2 definierten Anforderungen dargelegt. Beschrieben werden die Systemarchitektur mit den MV3-Kontexten in Abschnitt 4.1, das Interaktionskonzept in Abschnitt 4.2 und die Begründung der Technologieauswahl in Abschnitt 4.3.

4.1 Systemarchitektur

Die Systemarchitektur wird als kontextbasierte Architektur entworfen, die aus vier Ausführungskontexten einer Browser-Extension unter Manifest V3 zusammengesetzt wird, nämlich einem Popup-User-Interface, einem Background-Service, einem Content Script und einem Offscreen Document. In Abbildung 1 wird der Aufbau mit den Kommunikationswegen dargestellt. Als zentraler Bestandteil wird ein Background-Service vorgesehen, durch den die Kommunikation vermittelt und der Ablauf über eine Finite State Machine gesteuert wird. Weil in Manifest V3 Background Scripts als Service Worker ausgeführt werden, wird dem Background-Kontext kein direkter DOM-Zugriff gewährt, weshalb die Aufgaben verteilt werden.

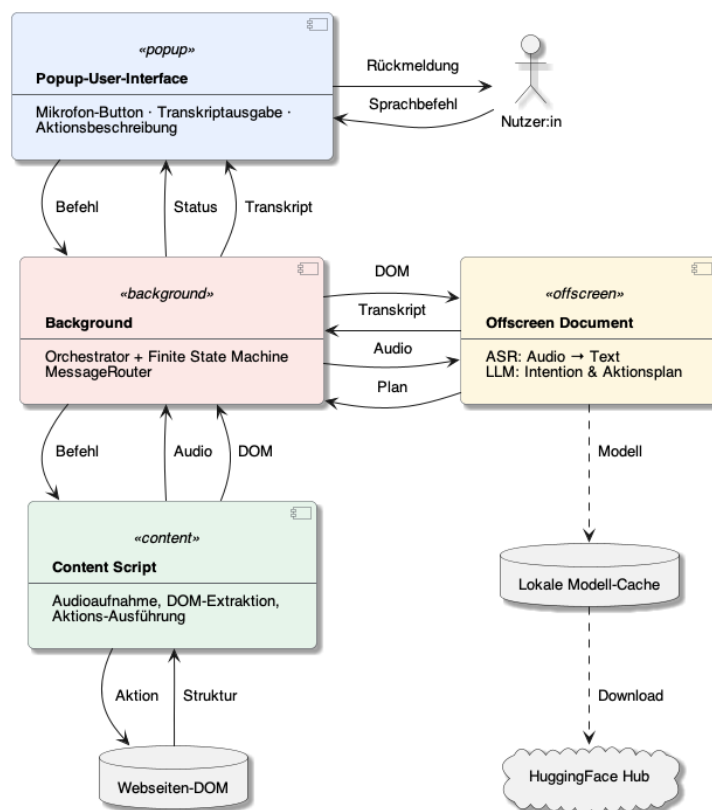


Abbildung 1: Systemarchitektur-Übersicht

Über das Popup-User-Interface werden Sprachbefehle eingegeben und der Systemstatus zurückgemeldet, gemäß FA-16 und FA-17 aus Tabelle 4. Ferner wird ein Aktionsprotokoll angezeigt, das durch FA-18 aus derselben Tabelle verlangt wird, Fehler werden verständlich gemäß FA-19 und FA-20 aus Tabelle 5 gemeldet und ein Erfolgsfeedback im Sinne von FA-22 aus derselben Tabelle gegeben. Aus gestalterischen Gründen wird eine durch NFA-5 aus Tabelle 7 minimalistisch gestaltete und zufolge NFA-4 aus derselben Tabelle tastaturbedienbare React-Oberfläche gewählt. Mit semantischen HTML-Strukturen und Accessible Rich Internet Applications (ARIA)-Beschriftungen wird die durch NFA-6 aus derselben Tabelle sowie NFA-16 aus Tabelle 10 geforderte Kompatibilität mit assistiven Technologien sichergestellt, und durch gezielte Farbwahl wird ein Kontrastverhältnis von mindestens WCAG AA erreicht, gemäß NFA-3 aus Tabelle 7.

Dem Background-Service wird die Rolle des Orchestrators übertragen, durch den Befehle weitergeleitet und Ergebnisse zurückgegeben werden. Damit der Zustand bei Seitenwechseln konsistent bleibt, wie durch NFA-15 aus Tabelle 10 gefordert wird, wird eine Finite State Machine eingesetzt, durch die der Verarbeitungsstatus deterministisch überführt wird. Da in Manifest V3 kein direkter Funktionsaufruf zwischen Kontexten möglich ist, wird die Kommunikation über einen MessageRouter abgewickelt [Google, 2024d]. Bei fehlerhaften Aktionen wird eine alternative Strategie ausgelöst, vorgesehen durch FA-21 aus Tabelle 5.

Über ein Content Script wird der direkte Zugang zur Webseite hergestellt, dabei die durch FA-01 aus Tabelle 1 geforderte Audioaufnahme durchgeführt, die DOM entsprechend FA-09 aus Tabelle 2 extrahiert und Aktionen ausgeführt, gemäß FA-11 aus Tabelle 3. Klick-, Text- und Auswahlaktionen, vorgesehen durch FA-12 und FA-13 aus Tabelle 3 sowie FA-08 aus Tabelle 2, laufen direkt auf den DOM-Elementen ab, wobei nach jeder Aktion die DOM aktualisiert und dem LLM erneut übergeben wird, was durch FA-15 aus derselben Tabelle verlangt wird. Die Audioaufnahme wird in das Content Script verlegt, weil dadurch der Zugriff auf die getUserMedia-API direkt im Seitenkontext möglich wird, und gestartet sowie gestoppt wird sie durch eine explizite Nutzeraktion, gemäß FA-04 aus Tabelle 1.

Für die KI-Komponenten wird ein Offscreen Document als Ausführungskontext vorgesehen, in dem die durch FA-02 und FA-03 aus Tabelle 1 verlangte Spracherkennung sowie die in FA-06 und FA-07 aus Tabelle 2 vorgesehene LLM-basierte Aktionsplanung vollständig lokal ablaufen. Vorrangig wird das Offscreen Document gewählt, weil es als einziger MV3-Kontext ein dauerhaftes, DOM-basiertes Ausführungsumfeld im Hintergrund bereitstellt [Google, 2024a]. Dadurch werden NFA-1 und NFA-2 aus Tabelle 6 erfüllt und nach initialem Download alle Funktionalitäten ohne Internetverbindung ausgeführt. Danach werden Modelle lokal gecacht, gefordert durch NFA-14 aus Tabelle 9, damit nur beim Erststart die durch NFA-10 aus derselben Tabelle vorgegebene Ladezeit von unter 30 Sekunden auftritt.

4.2 Interaktionskonzept

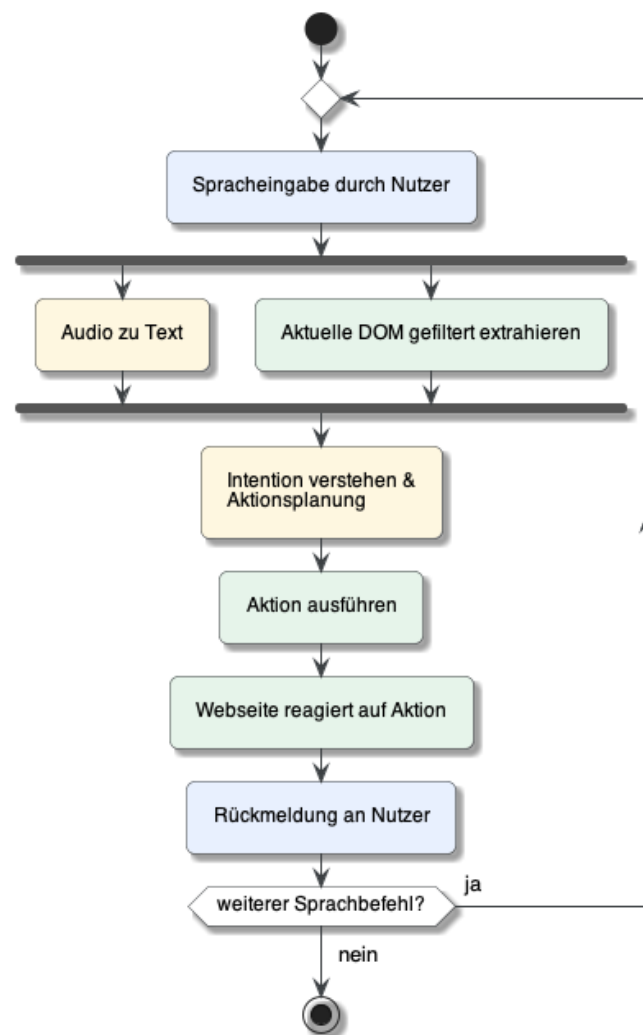
Veranschaulicht wird der Interaktionsablauf in Abbildung 2 als Aktivitätsdiagramm und in Abbildung 3 als Sequenzdiagramm.

Nach dem Öffnen des Popups wird der aktuelle Status durch den Background-Service abgefragt und der Mikrofon-Button aktiviert, womit die durch NFA-4 aus Tabelle 7 verlangte Tastaturbedienbarkeit sichergestellt wird. Manuell lässt sich die Aufnahme starten und stoppen, vorgesehen durch FA-04 aus Tabelle 1, und währenddessen wird der laufende Vorgang durch ein visuelles und akustisches Feedback gemeldet, gemäß FA-05 aus derselben Tabelle. Simultan wird das gesprochene Audio lokal in Text umgewandelt, gemäß FA-02 aus Tabelle 1, während gleichzeitig die aktuelle DOM der Webseite gefiltert extrahiert wird, was durch FA-09 aus Tabelle 2 gefordert wird. Aus Entwurfsgesichtspunkten werden Audio-Transkription und DOM-Extraktion parallel ausgeführt, wodurch sich die Latenz zwischen Spracheingabe und Aktionsbeginn verringert, gefordert durch NFA-11 aus Tabelle 9.

Sobald die Aufnahme stoppt, wird der Status auf *processing* gewechselt und das Transkript im Popup angezeigt, vorgesehen durch FA-17 aus Tabelle 4. Daraufhin wird durch das LLM im Offscreen Document die Intention erfasst und ein strukturierter Aktionsplan generiert, verlangt durch FA-06 und FA-07 aus Tabelle 2. Unterstützt werden dabei mindestens Klicken, Texteingabe, die Auswahl einer Option aus einem Dropdown und das Absenden eines Formulars, gefordert durch FA-08 aus derselben Tabelle. Vor der Ausführung wird der Plan im Status *planning* als Vorschau angezeigt, damit der Nutzer die Kontrolle behalten kann, was besonders für Nutzer mit motorischen Einschränkungen relevant ist. Mit unvollständigen oder mehrdeutigen Eingaben wird ein alternativer Plan abgeleitet, vorgesehen durch FA-10 aus Tabelle 2, und lässt sich kein verständlicher Plan generieren, wird eine Fehlermeldung ausgegeben, gemäß FA-20 aus Tabelle 5.

Während der Ausführung wird der Status auf *executing* gewechselt und die Aktionen werden sequentiell im Browser ausgeführt, festgelegt durch FA-11 aus Tabelle 3. Unmittelbar auf den identifizierten DOM-Elementen werden die durch FA-12 und FA-13 aus Tabelle 3 sowie FA-08 aus Tabelle 2 vorgesehenen Klick-, Text- und Auswahlaktionen ausgeführt. Ist die Ausführung abgeschlossen, wird der Status auf *ready* gewechselt und zusammen mit dem Ergebnis zurückgemeldet, vorgesehen durch FA-17 aus Tabelle 4 und FA-22 aus Tabelle 5. Zudem wird im Popup ein Aktionsprotokoll erzeugt, das durch FA-18 aus Tabelle 4 verlangt wird, und bei einem Fehlerstatus wird gegebenenfalls eine alternative Strategie ergriffen, gemäß FA-21 aus Tabelle 5.

Beim Erststart ohne Modelle wird der Download-Fortschritt im Popup angezeigt, wodurch sich die initiale Ladezeit transparent kommunizieren lässt, verlangt durch NFA-10 aus Tabelle 9. Sind die Modelle heruntergeladen, werden sie im lokalen Cache erhalten, vorgesehen durch NFA-14 aus Tabelle 9, damit alle Funktionalitäten ohne Internetverbindung ausgeführt werden können, gefordert durch NFA-2 aus Tabelle 6.



Farbe	MV3-Kontext	Rolle
	Popup	Barrierefreie React-UI
	Content Script	Mikrofon-Berechtigungen, DOM-Extraktion, Aktionsausführung
	Offscreen Document	Lokale ML via WebAssembly

Abbildung 2: Interaktionskonzept: Aktivitätsdiagramm

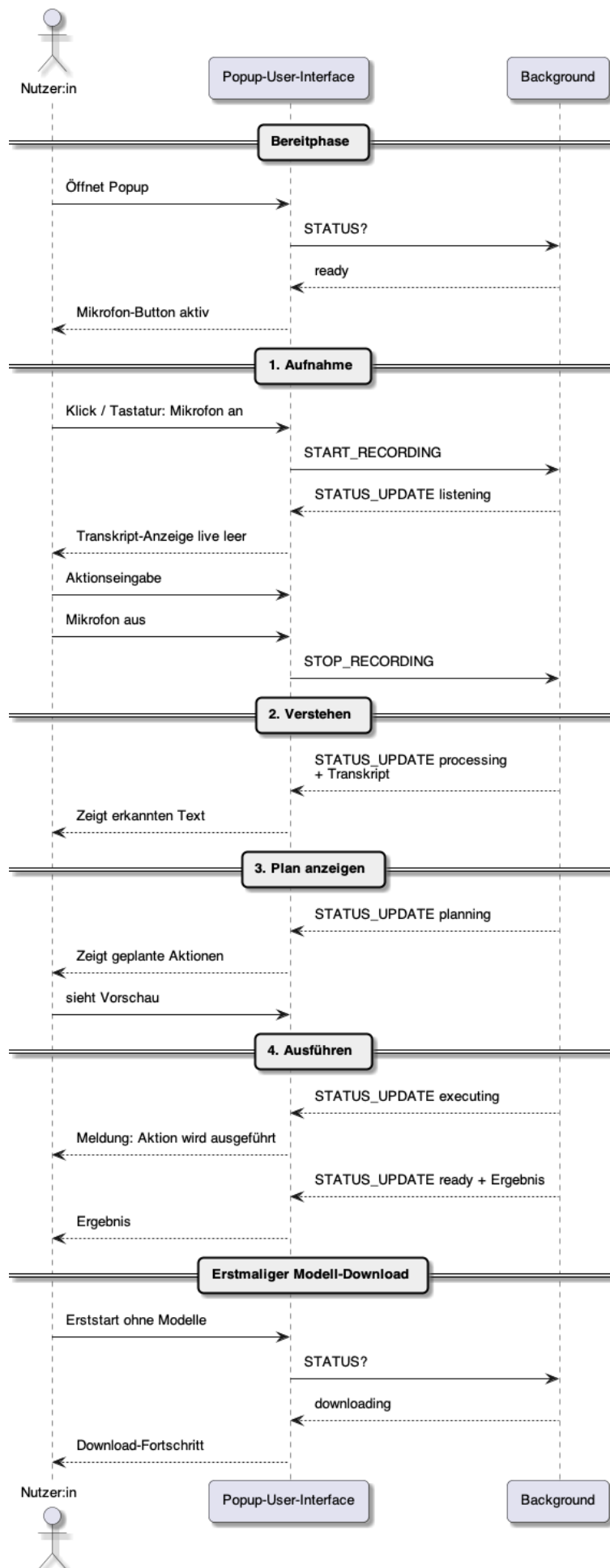


Abbildung 3: Interaktionskonzept: Sequenzdiagramm

4.3 Technologieauswahl

Grundlage der Extension ist das WebExtension-API unter Manifest V3, weil gegenüber Manifest V2 verbesserte Sicherheit, Leistung und Privatsphäre geboten und die browserübergreifende Lauf-fähigkeit unter Chrome, Firefox und Safari sichergestellt wird, verlangt durch NFA-7 bis NFA-9 aus Tabelle 8 [Pantelaïos and Kapravelos, 2024, S.2]. Allerdings wird durch Manifest V3 der Einsatz von Service Workern als Background-Scripts erzwungen, denen direkter DOM-Zugriff verwehrt bleibt, weshalb diese Einschränkung durch ein zur ML-Inferenz befähigtes Offscreen Document umgangen wird [Google, 2024a]. Eingesetzt wird außerdem das Framework WXT, weil eine vereinheitlichte, auf Vite basierende Entwicklungsumgebung für WebExtensions unter Manifest V3 bereitgestellt und Auto-Imports, Hot-Module-Replacement sowie eine browserübergreifende Build-Pipeline für Chrome, Firefox und Safari geboten werden, gefordert durch NFA-7 bis NFA-9 aus Tabelle 8 [WXT Project, 2024c]. Gegenüber einer manuellen Konfiguration wird WXT bevorzugt, weil die globalen Variablen `chrome` beziehungsweise `browser` zu einer einheitlichen `browser`-API zusammengeführt werden, wodurch sich browserübergreifender Code ohne bedingte Verzweigungen schreiben lässt, und Entrypoints für die vier Kontexte vereinfacht werden, wodurch sich die in Abschnitt 4.1 entworfenen Kontexte effizient umsetzen lassen.

Als technische Basis für das Popup-User-Interface werden React und WXT gemeinsam vorgesehen, weil über das `@wxt-dev/react`-Modul von WXT eine direkte Integration von React als Frontend-Framework ermöglicht wird. Statusänderungen lassen sich dank Reacts deklarativer User Interface (UI)-Beschreibung direkt an die Oberfläche binden, sodass das Feedback an den Nutzer verzögerungsfrei aktualisiert wird, vorgeschrieben durch FA-17 aus Tabelle 4.

Bei der DOM-Extraktion und -Interaktion wird die durch WXT bereitgestellte `browser`-API eingesetzt, vorgesehen durch FA-09 aus Tabelle 2 sowie FA-11 und FA-12 aus Tabelle 3. Automatisierungsframeworks wie Playwright oder Puppeteer werden aus Entwurfsgesichtspunkten außen vor gelassen, weil sie zusätzliche Berechtigungen und Prozesse erfordern, die den niedrigen Installationsaufwand erhöhen und potenziell mit assistiven Technologien kollidieren könnten, die durch NFA-16 aus Tabelle 10 geschützt werden sollen. Stattdessen werden die standardisierten DOM-APIs eingesetzt, wodurch Klick-, Text- und Auswahlaktionen direkt auf den identifizierten DOM-Elementen ausgeführt werden, vorgeschrieben durch FA-12 und FA-13 aus Tabelle 3 sowie FA-08 aus Tabelle 2.

Die lokale Spracherkennung wird durch Parakeet.js realisiert, wodurch das ASR-Modell Parakeet-TDT direkt im Browser über ONNX ausgeführt wird, ohne dass eine Cloud-Anbindung erforderlich ist, verlangt durch FA-02 aus Tabelle 1 und NFA-1 aus Tabelle 6 [ysdede, 2025] [Hugging Face, 2024b]. Wegen der durch FA-03 aus Tabelle 1 geforderten korrekten Transkription deutscher Spracheingaben wird Parakeet-TDT in der multilingualen Variante Version 3 gewählt, die durch quantisierte Gewichte im Browser inferiert wird [NVIDIA, 2024]. Durch willama wird die Inferenz des LLMs zur Aktionsplanung durchgeführt, weil dadurch eine WebAssembly-Anbindung an llama.cpp bereitgestellt wird, wodurch Modelle im GGUF-Format direkt im Browser ausgeführt werden können [ngxson, 2024]. Bevorzugt wird das GGUF-Format gegenüber dem ONNX-Format, weil dadurch eine größere Auswahl an LLMs auf HuggingFace verfügbar ist und eine direkte Kompatibilität mit llama.cpp besteht [ggml-org, 2024] [Hugging Face, 2024a]. Bewusst wird die Wahl auf ein quantisiertes LLM mit wenigen Milliarden Parametern getroffen, weil Quantisierung den Speicherbedarf und die Inferenzlatenz auf handelsüblicher Hardware reduziert, wodurch die Arbeitsspeichergrenze aus NFA-12 und die Latenz aus NFA-11, beide aus Tabelle 9, eingehalten werden [Shen et al., 2024, S.783] [Zheng et al., 2025, S.209]. Native Bibliotheken wie llama.cpp werden dank der WebAssembly-Runtime des Browsers plattformunabhängig im Browser ausgeführt, ohne dass zusätzliche Installationen nötig sind [Zhang et al., 2025, S.3].

Im anschließenden Kapitel 5 wird die Implementierung der in diesem Kapitel entworfenen Software mit konkreten Erklärungen des Quellcodes näher betrachtet.

5 Implementierung

In den nachfolgenden Abschnitten wird die Implementierung des in Kapitel 4 beschriebenen Entwurfs der Browser-Extension zur Beantwortung der Forschungsfrage erläutert. Dabei wird in Abschnitt 5.1 der Aufbau der Browser-Extension in der Code-Basis betrachtet. Anschließend wird die Audioaufnahme und Spracherkennung in Abschnitt 5.2 als erster Teil der Webseiten-Steuerungs-Pipeline erklärt. Wie daraus Aktionen mithilfe eines LLMs geplant werden, wird in Abschnitt 5.3 vertieft. In Abschnitt 5.4 wird erläutert, wie die geplanten Aktionen durch DOM-Interaktion ausgeführt werden. Abschließend wird die Fehlerbehandlung in Abschnitt 5.6 beleuchtet.

5.1 Aufbau der Browser-Extension

Für den Aufbau der Extension wird das in Abschnitt 4.3 begründete Framework WXT eingesetzt, wodurch der Quellcode im Verzeichnis `entrypoints/` in die fünf voneinander isolierten Ausführungskontexte `background`, `content`, `offscreen`, `popup` und `permission` gegliedert wird [WXT Project, 2024a]. Über die in Zeile 4 des Codeausschnitts 5.1 gezeigten Berechtigungen wird jeder Kontext abgesichert, und in den Zeilen 6 bis 9 wird eine `content_security_policy` mit `wasm-unsafe-eval` festgelegt, wodurch die WebAssembly-Ausführung erlaubt und die in Abschnitt 4.1 beschriebene Auslagerung der KI-Komponenten in ein Offscreen-Dokument ermöglicht wird, womit alle Daten lokal verarbeitet bleiben, wie in Tabelle 6 gefordert [WXT Project, 2024b] [Google, 2024c]. In den Zeilen 10 bis 18 wird ein Tastaturbefehl `toggle-recording` definiert, über den die Aufnahme unabhängig vom Popup gestartet wird, sodass alle Bedienelemente über die Tastatur erreichbar bleiben, wie in Tabelle 7 gefordert [Google, 2024b].

```
1 export default defineConfig({
2   modules: ['@wxt-dev/module-react'],
3   manifest: {
4     permissions: ['offscreen', 'activeTab', 'storage', 'unlimitedStorage'],
5     host_permissions: ['https://huggingface.co/*'],
6     content_security_policy: {
7       extension_pages:
8         "script-src 'self' 'wasm-unsafe-eval'; object-src 'self';",
9     },
10    commands: {
11      'toggle-recording': {
12        suggested_key: {
13          default: 'Ctrl+Shift+Space',
14          mac: 'Command+Shift+Space',
15        },
16        description: 'Start/Stop voice recording',
17      },
18    },
19  },
20 });
```

Codeausschnitt 5.1: Manifest-Konfiguration der Extension

5.1.1 Orchestrierung über Message-Router und Zustandsautomat

Als zentraler Knotenpunkt wird der Service Worker in `entrypoints/background/index.ts` über `defineBackground()` initialisiert, wobei eingehende Nachrichten einem generischen Message-Router aus `utils/messaging.ts` übergeben werden, dessen Funktion `createMessageRouter<T>()` in allen aktiven Kontexten identisch wiederverwendet wird, wie in Zeile 19 des Codeausschnitts 5.2 gezeigt wird [WXT Project, 2024a] [Microsoft Corporation, 2024]. Da in Manifest V3 nur asynchrone Nachrichten zwischen Kontexten ermöglicht werden, wird jede Antwort als aufgelöstes Promise zurückgegeben, das vom Aufrufer über `sendMessage()` abgewartet wird [Google, 2024d]. Der Verarbeitungsfortschritt wird in `entrypoints/background/state.ts` als endlicher Zustandsautomat gehalten, über `transitionTo()` gewechselt, über `persistState()` in `browser.storage.local` gesichert und über `broadcastStatus()` als `STATUS_UPDATE`-Nachricht an das Popup gesendet, wo die Hook `usePipelineStatus` aus Abschnitt 5.5 sie verarbeitet [WXT Project, 2024c].

```

1 export function createMessageRouter<T extends BaseMessage>(): MessageRouter<T> {
2   const handlers = new Map<string, Handler<T>>();
3
4   return {
5     addHandler(type, handler) {
6       handlers.set(type, handler as Handler<T>);
7     },
8
9     handle(msg) {
10      if (!isValidMessage(msg)) {
11        return undefined;
12      }
13
14      const handler = handlers.get((msg as BaseMessage).type);
15      if (!handler) {
16        return undefined;
17      }
18
19      return Promise.resolve(handler(msg as T));
20    },
21  };
22 }

```

Codeausschnitt 5.2: Generischer Message-Router aus `utils/messaging.ts`

5.1.2 Lazy-Initialisierung von Offscreen-Dokument und Berechtigungs-Tab

Weil einem Service Worker unter Manifest V3 kein DOM-Zugriff gewährt wird, wird das Offscreen-Dokument über `setupOffscreenDocument()` in `offscreen-api.ts` bedarfsgesteuert statt dauerhaft erzeugt, wobei über ein `offscreenDocumentCreating`-Flag und `waitForOffscreenCreation()` ein Erzeugungsrennen unter parallelen Aufrufen vermieden wird [Google, 2024a]. Da die Aufnahme nur nach expliziter Berechtigungserteilung unterstützt wird, wie in Tabelle 1 gefordert, wird sie zunächst direkt im Offscreen-Dokument angefordert, was ohne bereits erteilte Berechtigung abgelehnt wird. Wird ein solcher Fehlschlag über `isMicPermissionError()` erkannt, wird über `openPermissionTab()` ein fünfter Kontext als regulärer Browser-Tab mit `permission.html` geöffnet, wie in Zeile 3 des Codeausschnitts 5.3 dargestellt, wobei die erteilte Berechtigung dank gemeinsamen `chrome-extension://`-Ursprungs auch dem Offscreen-Dokument zur Verfügung steht und eine erneute Aufnahme ohne weiteren Dialog ermöglicht.

```

1 async function requestPermission(): Promise<void> {
2   try {
3     const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
4     stream.getTracks().forEach((track) => track.stop());
5     messageEl.textContent =
6       'Microphone access granted. This tab will close automatically.';
7     setTimeout(() => window.close(), 1500);
8   } catch (err) {
9     messageEl.textContent = 'Microphone access was not granted (${String(err)}). Please click
10      "Try again" and allow access.';
11     retryButton.style.display = 'inline-block';
12   }
13 }

```

Codeausschnitt 5.3: Einmalige Berechtigungsanfrage in `entrypoints/permission/main.ts`

5.2 Audioaufnahme und Spracherkennung

In diesem Abschnitt wird die Verarbeitung der Spracheingabe von der Audioaufnahme bis zur nutzbaren Textausgabe beschrieben, gegliedert in die Aufnahme in Abschnitt 5.2.1, die lokale Transkription in Abschnitt 5.2.2 und die Konfidenzbewertung in Abschnitt 5.2.3.

5.2.1 Aufnahme im Offscreen-Dokument

Die Audioaufnahme wird im Offscreen-Dokument durchgeführt, wodurch der Mikrofonzugriff auf die Browser-Extension beschränkt wird. Über die Funktion `startRecording()` in `entrypoints/offscreen/recorder.ts` wird ein Mikrofon-Datenstrom angefordert, einem `MediaRecorder` übergeben und nach Beendigung im `onstop`-Handler als Blob über `transcribeAudio()` zur Transkription weitergegeben, wie in den Zeilen 3 bis 4 und 19 bis 26 des Codeausschnitts 5.4 gezeigt [Mozilla, 2024f] [Mozilla, 2024g].

```

1 export async function startRecording(): Promise<{ success: boolean; error?: string }> {
2   recordedChunks = [];
3   mediaStream = await navigator.mediaDevices.getUserMedia({ audio: true });
4   mediaRecorder = new MediaRecorder(mediaStream, { mimeType });
5   const recorder = mediaRecorder;
6
7   const mimeType = MediaRecorder.isTypeSupported('audio/webm;codecs=opus')
8     ? 'audio/webm;codecs=opus'
9     : MediaRecorder.isTypeSupported('audio/webm')
10      ? 'audio/webm'
11      : 'audio/mp3';
12
13   recorder.ondataavailable = (event) => {
14     if (event.data.size > 0) {
15       recordedChunks.push(event.data);
16     }
17   };
18
19   recorder.onstop = async () => {
20     const blob = new Blob(recordedChunks, { type: recorder.mimeType });
21     const arrayBuffer = await blob.arrayBuffer();
22     await transcribeAudio(
23       Array.from(new Uint8Array(arrayBuffer)),
24       recorder.mimeType
25     );
26   };
27
28   recorder.start(100);
29   return { success: true };
30 }

```

Codeausschnitt 5.4: Aufnahmestart in `entrypoints/offscreen/recorder.ts`

5.2.2 Lokale Transkription

Für die Transkription wird ausschließlich die Bibliothek `parakeet.js` eingesetzt, wodurch keine eigene Erkennungslogik entwickelt wird [ysdede, 2025]. Das multilinguale Parakeet-TDT-Modell in Version 3 wird erst beim ersten Aufruf verzögert geladen und über ONNX Runtime Web ausgeführt, wobei der Audio-Puffer in mono-kanaliges 16-kHz-Pulse-Code Modulation (PCM) konvertiert wird [Microsoft, 2024] [NVIDIA, 2024]. Die Transkription wird über `model.transcribeLongAudio()` angestoßen, deren Ergebnis als `RECORDING_RESULT`-Nachricht an den Background-Service übermittelt wird, wo die Weiterverarbeitung in Abschnitt 5.3 erklärt wird [ysdede, 2025].

5.2.3 Konfidenzbewertung und Nutzer-Feedback

Aus der von `model.transcribeLongAudio()` zurückgegebenen Konfidenz wird über `computeOverallConfidence()` ein Durchschnittswert gebildet. Wird der Schwellenwert `lowConfidenceThreshold` von 0,6 unterschritten oder kein Text erkannt, wird eine Warnung erzeugt und mit dem Transkript übermittelt, die im Popup angezeigt wird, wie in Tabelle 5 gefordert. Über `logAsrMetrics()` wird ein Echtzeitfaktor aus Erkennungsdauer und Audiolänge erfasst, womit die in Tabelle 9 geforderte Latenz messbar gespeichert wird für die Evaluierungen.

5.3 Aktionsplanung mit dem lokalen LLM

Weil bei der Aktionsplanung zwei lokale Modelle gegen das DOM einer realen Webseite orchestriert werden und von einem 1,5-Milliarden-Parameter-Modell zuverlässig geplant werden muss, wird jede Stelle, an der dem Modell vertraut wird, früher oder später zum Fehler, weshalb als wiederkehrende Lösung jede Modell-Aussage validiert und bei Versagen deterministisch auf einen Fallback zurückgegriffen wird. Dieser Abschnitt wird in das Laden des Modells mit grammatikgestützter Generierung in Abschnitt 5.3.1, die Serialisierung der DOM-Elemente in Abschnitt 5.3.2, das Prompt-Engineering in Abschnitt 5.3.3, die Antwort-Verarbeitung und Validierung in Abschnitt 5.3.4, die Replan-Schleife mit Konvergenzsicherungen in Abschnitt 5.3.5 und den Regressionsverlauf in Abschnitt 5.3.6 gegliedert.

5.3.1 Modell-Laden und grammatikgestützte Generierung

In `entrypoints/offscreen/action-planner/model.ts` wird dasselbe verzögerte Ladeprinzip wie beim Transkriptionsmodell verfolgt, wobei das quantisierte Qwen2.5-1.5B-Instruct-Modell im GGUF-Format geladen wird [ggml-org, 2024] [Hugging Face, 2024a] [Zheng et al., 2025, S.209]. Damit ein bereits heruntergeladenes Modell auch ohne Internetverbindung aus dem Browser-Cache geladen wird, wird die `Wllama`-Instanz mit `allowOffline: true` initialisiert, womit ein Offline-Betrieb gewährleistet wird, wie in Tabelle 6 gefordert [ngxson, 2024]. Über den Parameter `grammar` wird eine selbst verfasste GGML Backus-Naur Form (GBNF)-Grammatik an `createChatCompletion()` übergeben, wie in Zeile 6 des Codeausschnitts 5.5 zu sehen, wodurch der Ausgaberaum auf syntaktisch gültiges JavaScript Object Notation (JSON) eingeschränkt wird, während bei einer Temperatur von 0,1 in Zeile 4 eine hohe Determiniertheit angestrebt wird [ngxson, 2024].

Weil `wllama` nicht nebenläufigkeitssicher ist, werden Inferenzaufrufe über `runExclusive()` in eine Warteschlange serialisiert, wodurch gleichzeitige Anforderungen nicht zum Absturz führen [ngxson, 2024].

```

1 return await runExclusive(() =>
2   wllamaInstance!.createChatCompletion({
3     messages,
4     temperature: 0.1,
5     max_tokens: maxTokens,
6     grammar: JSON_GRAMMAR,
7     stream: false,
8     abortSignal: abortController.signal,
9   })
10 );

```

Codeausschnitt 5.5: Grammatikgestützte Generierung in `entrypoints/offscreen/action-planner/model.ts`

5.3.2 DOM-Serialisierung und Kontextreduktion

Weil das Kontextfenster des lokalen Sprachmodells begrenzt ist, wird die vollständige DOM-Struktur einer Webseite vor der Weitergabe gefiltert und komprimiert, wozu sie in `serializeDOMElements()` in eine kompakte Textform überführt wird. Jedes Element wird anhand der Überschneidung von Transkript-Schlagwörtern mit Kennung, Platzhaltertext und Beschriftung bewertet. Über `selectTopElements()` in `entrypoints/offscreen/action-planner/prompt.ts` wird bewertet und absteigend sortiert, wie in Codeausschnitt 5.6 dargestellt, wobei zwischen `MIN_DOM_ELEMENTS` (6) und `MAX_DOM_ELEMENTS` (50) die passende Anzahl beibehalten wird.

```

1 export function selectTopElements(
2   elements: DOMElement[],
3   transcript: string
4 ): DOMElement[] {
5   const filteredKws = extractFilteredKeywords(transcript);
6   const impliesFill = commandImpliesFill(transcript);
7
8   const scored = elements
9     .filter((el) => el.id || el.label || el.placeholder || el.name || el.text || el.type ===
10       'submit' || el.type === 'button')
11     .map((el) => {
12       const hay = [el.id, el.label, el.placeholder, el.name, el.text, el.type]
13         .filter(Boolean).join(' ').toLowerCase();
14       const keywordScore = filteredKws.reduce(
15         (s, kw) => s + (keywordMatches(kw, hay, tokenize(hay)) ? 1 : 0), 0);
16       const viewportBonus = el.inViewport ? 0.3 : 0;
17       const fillableBonus = impliesFill && isFillableInput(el) ? 1000 : 0;
18       return { el, score: keywordScore + viewportBonus + fillableBonus };
19     })
20     .sort((a, b) => b.score - a.score || a.el.index - b.el.index);
21
22   const keep = Math.min(MAX_DOM_ELEMENTS, Math.max(MIN_DOM_ELEMENTS, scored.length));
23   return scored.slice(0, keep).map((s) => s.el);

```

Codeausschnitt 5.6: Element-Ranking in `entrypoints/offscreen/action-planner/prompt.ts`

In Zeile 16 des Codeausschnitts 5.6 wird dem füllbaren Eingabefeld bei einem Fill-implizierenden Befehl ein Bonus von 1000 zugeschlagen, damit das echte Eingabefeld im Prompt vor keywordstärkeren Ankern steht, was aus einem Regressionsfall in Abschnitt 5.3.6 hervorgeht, bei dem ein Bonus von 0,5 nicht ausreichte. Über `buildHistoryText()` wird den bereits ausgeführten Aktionen ein Verlaufstext vorangestellt, der das Modell davor warnt, alte Selektoren wiederzuverwenden, weil die DOM-Liste nach jeder Seiteaktualisierung neu zugewiesen wird.

5.3.3 Prompt-Engineering und Aktionshinweise

Der in `entrypoints/offscreen/action-planner/prompt.ts` selbst verfasste Systemprompt enthält acht nummerierte Anweisungen. Jeder DOM-Zeile wird über `actionHintFor()` ein Aktionshinweis `action=fill`, `action=select` oder `action=click` beigefügt, wodurch das Modell bei der Typwahl geführt wird, wobei über `isFillableInput()` nur textartigen Eingabefeldern der Fill-Hinweis zugewiesen wird, während `submit`, `button`, `image`, `reset`, `checkbox` und `radio` den Click-Hinweis erhalten. Diese Trennung wird eingeführt, weil ein Submit-Button als `action=fill` annotiert wird und das Modell dadurch systematisch zum falschen Typ verführt wird. Weil bei Verbundbefehlen mehrere Schritte auf einmal geplant werden, deren spätere Elemente noch nicht im DOM sind, wird in Schritt 5 des Systemprompts ausdrücklich gefordert, dass ausschließlich der erste Schritt geplant wird, weil nach jeder Seitenaktualisierung erneut nachgefragt wird. Um den Ausgaberaum auf die unterstützten Aktionstypen zu beschränken, wird der Systemprompt zusätzlich aus den in `utils/actions.ts` gepflegten `AVAILABLE_ACTIONS` gespeist, die auch in die Ausführung im Content Script einfließen, wodurch eine Abweichung zwischen geplanten und ausführbaren Aktionen strukturell ausgeschlossen wird, wie in Tabelle 3 gefordert.

5.3.4 Antwort-Verarbeitung und Validierung

Weil lokale Modelle Antworten unvollständig oder mit repetitiven Tokenfolgen abbrechen, wird der Text über `extractJSONBlock()` und `safeJSONParse()` aus `entrypoints/offscreen/action-planner/response-parser.ts` in einen strukturierten Aktionsplan überführt, wie in den Codeausschnitten 5.7 und 5.8 gezeigt wird. In den Zeilen 6 bis 9 des Codeausschnitts 5.7 wird bei fehlender schließender Klammer das letzte vollständige Aktionsobjekt gesucht und die Struktur durch Anhängen von `}}` rekonstruiert, wodurch eine abgebrochene Antwort dennoch verwertbar wird.

```

1 export function extractJSONBlock(text: string): string | null {
2   const firstBrace = text.indexOf('{');
3   if (firstBrace === -1) return null;
4
5   const lastBrace = text.lastIndexOf('}');
6   if (lastBrace === -1 || lastBrace <= firstBrace) {
7     const lastCompleteAction = text.lastIndexOf('}');
8     if (lastCompleteAction > firstBrace) {
9       return text.slice(firstBrace, lastCompleteAction + 1) + '}}';
10    }
11    return null;
12  }
13  return text.slice(firstBrace, lastBrace + 1);
14 }
```

Codeausschnitt 5.7: Extraktion des JSON-Blocks in `entrypoints/offscreen/action-planner/response-parser.ts`

Schlägt das reguläre `JSON.parse` fehl, wird in den Zeilen 5 bis 15 des Codeausschnitts 5.8 über einen regulären Ausdruck eine einzelne Aktion extrahiert und ein `recoveredFromTruncation`-Flag gesetzt, das in Abschnitt 5.3.5 eine weitere Planungsrunde erzwingt [Mozilla, 2024e]. Ein als `finish_reason === 'length'` erkannter Längenabbruch wird ebenfalls als Replan-Grund gewertet, wodurch unvollständige Antworten nicht unbemerkt akzeptiert werden.

```

1 export function safeJSONParse(text: string): PlannedActionsJSON | null {
2   try {
3     return JSON.parse(text) as PlannedActionsJSON;
4   } catch {
5     const actionMatch = text.match(
6       /"type"\s*:\s*"(\w+)"[\s\S]*?"selector"\s*:\s*"([~"]+)" /
7     );
8     if (actionMatch) {
9       const action: DOMAction = {
10         type: actionMatch[1] as DOMAction['type'],
11         selector: actionMatch[2],
12       };
13       const valueMatch = text.match(/"value"\s*:\s*"([~"]+)" /);
14       if (valueMatch) action.value = valueMatch[1];
15       return { actions: [action], recoveredFromTruncation: true };
16     }
17     return null;
18   }
19 }

```

Codeausschnitt 5.8: Sicheres JSON-Parsing in `entrypoints/offscreen/action-planner/response-parser.ts`

Anschließend wird der Plan über `validatePlannedActions()` in `entrypoints/offscreen/action-planner/validation.ts` gegen das angebotene DOM geprüft, wie in Codeausschnitt 5.9 dargestellt. In den Zeilen 16 bis 19 wird eine Map von Selektoren auf Elemente aufgebaut, gegen die jeder geplante Selektor in Zeile 32 aufgelöst wird, womit erfundene Selektoren erkannt und verworfen werden. Über `coerceType()` in den Zeilen 50 bis 60 wird der Aktionstyp an das Zielelement angepasst, wobei `select` ausschließlich auf `<select>` erlaubt und ein `fill` auf einem `<select>` zu `select` korrigiert wird, während `fill` auf nicht füllbaren Elementen abgelehnt wird. Die Zähler `dropped`, `unknownSelector` und `typeMismatch` werden zurückgegeben und speisen den `replanable`-Pfad, sodass ungültige Pläne einen Replan auslösen statt stillschweigend ausgeführt zu werden.

```

1 export function validatePlannedActions(
2   actions: DOMAction[],
3   offered: DOMELEMENT[]
4 ): { actions: DOMAction[]; dropped: number; unknownSelector: number; typeMismatch: number } {
5   const bySelector = new Map<string, DOMELEMENT>();
6   for (const el of offered) {
7     bySelector.set(normalizeSelector(el.selector), el);
8   }
9
10  const valid: DOMAction[] = [];
11  let dropped = 0, unknownSelector = 0, typeMismatch = 0;
12
13  for (const action of actions) {
14    if (!action || !action.selector || !VALID_TYPES.has(action.type)) {
15      dropped++; typeMismatch++; continue;
16    }
17    const el = bySelector.get(normalizeSelector(action.selector));
18    if (!el) { dropped++; unknownSelector++; continue; }
19    const coerced = coerceType(action, el);
20    if (!coerced) { dropped++; typeMismatch++; continue; }
21    valid.push(coerced);
22  }
23  return { actions: valid, dropped, unknownSelector, typeMismatch };
24 }
25
26 function coerceType(action: DOMAction, el: DOMELEMENT): DOMAction | null {
27   const type = action.type;
28   if (type === 'select' && el.tag !== 'select') return null;
29   if (type === 'fill') {
30     if (el.tag === 'select') return { ...action, type: 'select' };
31     if (!isFillableInput(el)) return null;
32   }
33   return action;
34 }

```

Codeausschnitt 5.9: Validierung und Typ-Korrektur in `entrypoints/offscreen/action-planner/validation.ts`

5.3.5 Replan-Schleife mit Konvergenzsicherungen

Weil ein Klick oder ein Formularabsenden die Webseite verändern kann, wird diese Entscheidung nicht dem Modell allein überlassen. Mit `truncateAtFirstDomChange()` in `entrypoints/background/handlers/action-planning-utils.ts` wird jeder Plan nach der ersten DOM-verändernden Aktion abgeschnitten, wie in Codeausschnitt 5.10 dargestellt, unabhängig davon, was das Modell als `done` zurückmeldet. In Zeile 2 wird `click` als DOM-verändernd festgelegt, und in den Zeilen 7 bis 11 wird der Plan vor der ersten solchen Aktion abgeschnitten, falls noch weitere folgen, wodurch Aktionen gegen veraltetes DOM verworfen werden.

```

1 const DOM_CHANGING_ACTION_TYPES = new Set(['click']);
2
3 export function truncateAtFirstDomChange(actions: DOMAction[]): {
4   toExecute: DOMAction[]; truncated: boolean;
5 } {
6   const idx = actions.findIndex((a) => DOM_CHANGING_ACTION_TYPES.has(a.type));
7   if (idx === -1 || idx === actions.length - 1) {
8     return { toExecute: actions, truncated: false };
9   }
10  return { toExecute: actions.slice(0, idx + 1), truncated: true };
11 }

```

Codeausschnitt 5.10: Deterministisches Abschneiden in `action-planning-utils.ts`

Nach der Ausführung wird über `requestNextPlan()` das DOM erneut vom Content Script angefordert, wobei über `waitForDomSettle: executedActions.length > 0` sichergestellt wird, dass erst eine stabile DOM-Struktur abgewartet wird, bevor erneut extrahiert wird. Zusammen mit den bereits ausgeführten Aktionen wird das frische DOM erneut an das Modell übergeben, wodurch wiederholte Planungsrunden gegen den jeweils aktuellen Seitenzustand entstehen. Weil asynchrone Modellinferenz und Service-Worker-Restarts Race-Conditions erzeugen, wird über einen monotonen Zähler in `state.ts` jede neue Pipeline-Ausführung inkrementiert, wie in Zeile 8 des Codeausschnitts 5.11 gezeigt, und jeder asynchrone Rückkehrer wird gegen den aktuellen Zählerstand geprüft, wodurch veraltete Ergebnisse verworfen werden. Über `MAX_REPLAN_ITERATIONS` von fünf wird die Anzahl der Runden begrenzt, und über eine Signatur `planSignature()` wird erkannt, wenn ein wiederholter Plan dem vorherigen entspricht, sowie über `lastRejectedSignature`, wenn ein abgelehnter Plan erneut abgelehnt wird, wonach der Ablauf jeweils mit einer Nutzerwarnung beendet wird, wie in den Zeilen 8 bis 11 des Codeausschnitts 5.12 zu sehen.

```

1 let generation = 0;
2
3 export function getGeneration(): number {
4   return generation;
5 }
6
7 export function bumpGeneration(): number {
8   generation += 1;
9   return generation;
10 }

```

Codeausschnitt 5.11: Generation-Zähler in `entrypoints/background/state.ts`

```

1 export async function handleActionsPlanned(msg: ActionsPlannedMessage): Promise<void> {
2   if (planningGeneration !== getGeneration()) return;
3   clearPlanningWatchdog();
4
5   if (msg.error) {
6     const rejectedSignature = msg.plannedActions && msg.plannedActions.length > 0
7       ? planSignature(msg.plannedActions) : undefined;
8     if (rejectedSignature !== undefined
9       && lastRejectedSignature !== undefined
10      && rejectedSignature === lastRejectedSignature) {
11       resetPlanningState(); await transitionTo('ready'); return;
12     }
13     lastRejectedSignature = rejectedSignature;
14     if (msg.replanable && validationRetries < MAX_VALIDATION_RETRIES
15       && replanStep < MAX_REPLAN_ITERATIONS) {
16       validationRetries += 1; replanStep += 1;
17       await requestNextPlan(planningTabId, getTranscript(),
18         executedActionsAccumulator, replanStep + 1, 'validation_retry');
19       return;
20     }
21   }
22   const { toExecute, truncated } = truncateAtFirstDomChange(msg.actions);
23   const signature = planSignature(toExecute);
24   if (lastPlanSignature !== undefined && signature === lastPlanSignature) {
25     resetPlanningState(); await transitionTo('ready'); return;
26   }
27   lastPlanSignature = signature;
28 }

```

Codeausschnitt 5.12: Ausschnitt aus dem Replan-Entscheidungsknoten in `entrypoints/background/handlers/action-planning.ts`

In Zeile 2 des Codeausschnitts 5.12 wird der veraltete Plan verworfen, sobald die erfasste Generation nicht mehr der aktuellen entspricht. Die beiden Wiederholungserkennungen in den Zeilen 8 bis 11 und 24 bis 25 brechen die Schleife ab, sobald dasselbe Ergebnis erneut auftritt, womit eine Endlosschleife vermieden wird, die ohne Wiederholungsprüfung allein durch `Replan-on-Failure` entstünde. Ergänzend fängt ein Watchdog mit `PLANNING_TIMEOUT_MS` von 60s ein Hängen des lokalen Modells ab, wobei über `abortPlanning()` ein `AbortController` an `createChatCompletion()` weitergereicht wird, wie in Zeile 8 des Codeausschnitts 5.5 zu sehen [ngxson, 2024] [Mozilla, 2024a]. Über `summarizeFailedActions()`, `allFailuresRecoverable()` und `successfulActions()` wird nach der Ausführung unterschieden, ob bei wiederherstellbaren Fehlern erneut geplant oder der Ablauf beendet wird, wodurch mehrschrittige Eingaben schrittweise abgearbeitet werden, wie in Tabelle 2 gefordert.

5.3.6 Regressionsverlauf und deterministischer Fallback

Nach Einführung der vorgenannten Sicherungen wird eine Regression beobachtet, bei der auf keiner Seite ein Befehl mehr ausgeführt wird und nach abgelehnten Plänen mit anschließendem Replan derselbe Fehler reproduziert wird. Dabei werden zwei verschränkte Modi beobachtet. Im ersten Modus wird das Eingabefeld im Prompt korrekt mit `action=fill` erkannt, dennoch wird nach Textrelevanz ein keyword-reicher Anker statt des echten Eingabefelds gewählt, woraufhin das `fill` auf dem Anker durch `coerceType()` deterministisch abgelehnt wird, wodurch ein Replan ausgelöst wird, bei dem derselbe Fehler reproduziert wird. Im zweiten Modus werden bei wenigen Elementen durch das 1,5-Milliarden-Parameter-Modell repetitive Tokenfolgen erzeugt, bis die Token-Begrenzung erreicht wird, woraufhin ein Replan in dieselbe Degeneration ausgelöst wird. Als Gegenmaßnahme wird ein dreistufiges deterministisches Fallback-Verfahren umgesetzt.

Erstens wird über die deterministische Funktion `buildFallback()` in `entrypoints/offscreen/action-planner/prompt.ts`, wenn ein Fill-Befehl vorliegt und der Modellplan nicht verwertbar ist, der Fill-Wert per regulärem Ausdruck aus dem Transkript extrahiert, das oberste füllbare Eingabefeld und der bestbewertete Submit-Button ausgewählt, wie in Codeausschnitt 5.13 gezeigt, wodurch die Pipeline auch bei Modellversagen deterministisch fortgesetzt wird.

```

1 export function buildFallback(
2   transcript: string,
3   offered: DOMElement[]
4 ): { actions: DOMAction[]; done: boolean } | null {
5   if (!commandImpliesFill(transcript)) return null;
6   const value = extractFillValue(transcript);
7   if (!value) return null;
8   const input = offered.find(isFillableInput);
9   if (!input) return null;
10  const actions: DOMAction[] = [{ type: 'fill', selector: input.selector, value }];
11  const submit = findSubmitButton(offered);
12  if (submit) actions.push({ type: 'click', selector: submit.selector });
13  return { actions, done: !!submit };
14 }

```

Codeausschnitt 5.13: Deterministischer Fallback-Planer in `entrypoints/offscreen/action-planner/prompt.ts`

In den Zeilen 5 bis 7 des Codeausschnitts 5.13 wird geprüft, ob der Befehl Fill impliziert und ein Wert extrahierbar ist, und in Zeile 8 wird das erste füllbare Eingabefeld bestimmt, wohingegen in den Zeilen 11 bis 12 der Submit-Button ergänzt wird. Zweitens wird der `fillableBonus` von 0,5 auf 1000 angehoben, wie bereits in Zeile 16 des Codeausschnitts 5.6 gezeigt, damit das echte Eingabefeld im Prompt vor keyword-stärkeren Ankern steht. Drittens wird der Systemprompt dahingehend geschärft, dass eine `action=click`-Zeile niemals ein Fill-Ziel ist und das Fill-Ziel stets ein `<input>` oder `<textarea>` mit `#id`- oder `[name=...]`-Selektor ist. Über Telemetrie wird der Effekt belegt, wonach die Replan-Iterationen von bis zu fünf auf überwiegend eine und die gesamte Pipeline-Dauer von 156s auf 118s zurückgehen. Eine ungelöste Einschränkung bleibt durch die kleine Modellarchitektur bedingt, wodurch bei sehr komplexen Verbundbefehlen dennoch Wiederholungs-Schleifen auftreten können.

5.4 DOM-Interaktion

In diesem Abschnitt wird erläutert, wie die geplanten Aktionen durch direkte DOM-Interaktion aus dem Content Script ausgeführt werden. Über `extractDOMElements()` in `entrypoints/content/dom-extractor.ts` wird eine handverfasste Liste interaktiver Selektoren abgefragt und jedes Element über eine mehrstufige Sichtbarkeitsprüfung gefiltert, die Null-Größe, `aria-hidden` und `checkVisibility` abweist. Jedem Element wird über `buildSelector()` ein stabiler Selektor zugewiesen, wobei einer dreistufigen Fallback-Strategie gefolgt wird, wie in Codeausschnitt 5.14 dargestellt, in der in den Zeilen 2 und 3 bevorzugt eine `#id` verwendet wird, in den Zeilen 4 bis 8 ein `[name=...]`-Selektor nur bei Eindeutigkeit im Dokument, und in den Zeilen 9 bis 11 ein synthetisches `data-lava-idx`-Attribut injiziert wird, wodurch auch ansonsten unselektierbare Elemente adressierbar werden.

```

1 function buildSelector(el: Element, name: string | undefined, index: number): string {
2   if (el.id) return `#${CSS.escape(el.id)}`;
3   if (name) {
4     const escaped = CSS.escape(name);
5     if (document.querySelectorAll(`[name="${escaped}"]`).length === 1) {
6       return `[name="${escaped}"]`;
7     }
8   }
9   const syntheticValue = `i${index}`;
10  el.setAttribute(SYNTHETIC_SELECTOR_ATTR, syntheticValue);
11  return `[${SYNTHETIC_SELECTOR_ATTR}=${syntheticValue}]`;
12 }

```

Codeausschnitt 5.14: Selektor-Vergabe in `entrypoints/content/dom-extractor.ts`

Zur Ausführung wird über `executeActions()` in `entrypoints/content/action-executer.ts` je Aktion das Ziel über `normalizeSelector()` aufgelöst, validiert und an den passenden Executor delegiert. Weil reaktive Frameworks wie React, Vue und Svelte ein naives Setzen von `.value` ignorieren, wird über `setNativeValue()` der in den Zeilen 7 bis 9 des Codeausschnitts 5.15 gezeigte Setter aus dem Prototyp-Deskriptor verwendet, wonach ein `InputEvent` und ein `change`-Event ausgelöst werden, wodurch die Framework-Reaktivität ausgelöst wird und die Suche tatsächlich startet [Mozilla, 2024i]. Jede Aktion wird in einen `try/catch`-Block gekapselt, sodass ein einzelner Fehlschlag nicht den gesamten Batch abbricht, und jedem Fehlschlag wird eine Fehlerkategorie wie `element_not_found`, `element_not_visible`, `element_disabled` oder `executor_error` zugeordnet, die in Abschnitt 5.3.5 die Replan-Entscheidung speist.

```

1 function setNativeValue(el: HTMLInputElement | HTMLTextAreaElement, value: string): void {
2   const proto = el instanceof HTMLTextAreaElement
3     ? HTMLTextAreaElement.prototype
4     : HTMLInputElement.prototype;
5   const descriptor = Object.getOwnPropertyDescriptor(proto, 'value');
6   if (descriptor?.set) descriptor.set.call(el, value);
7   else el.value = value;
8 }

```

Codeausschnitt 5.15: Framework-kompatibler Wert-Setter in `entrypoints/content/action-executer.ts`

```

1 export function waitForDomSettle(idleMs = 250, timeoutMs = 1500): Promise<void> {
2   return new Promise<void>((resolve) => {
3     const target = document.body;
4     if (!target) { resolve(); return; }
5     let idleTimer, hardTimer; let settled = false;
6     const finish = () => {
7       if (settled) return; settled = true;
8       if (idleTimer) clearTimeout(idleTimer);
9       clearTimeout(hardTimer); observer.disconnect(); resolve();
10    };
11    const scheduleIdle = () => {
12      if (idleTimer) clearTimeout(idleTimer);
13      idleTimer = setTimeout(finish, idleMs);
14    };
15    const observer = new MutationObserver(() => scheduleIdle());
16    observer.observe(target, { childList: true, subtree: true,
17      attributes: true, characterData: true });
18    scheduleIdle();
19    hardTimer = setTimeout(finish, timeoutMs);
20  });
21 }

```

Codeausschnitt 5.16: DOM-Beruhigung in `entrypoints/content/dom-settle.ts`

Damit bei einem Replan nicht gegen eine noch instabile DOM-Struktur geplant wird, wird über `waitForDomSettle()` in `entrypoints/content/dom-settle.ts` ein `MutationObserver` auf `document.body` angesetzt, wie in Zeile 16 des Codeausschnitts 5.16 gezeigt, der bei jeder Mutation einen Leerlauf-Timer zurücksetzt und erst auflöst, wenn die Struktur für 250 ms ruhig bleibt oder ein harter Timeout von 1500 ms erreicht wird [Mozilla, 2024h].

5.5 Popup-User-Interface

Mit dem Entrypoint `entrypoints/popup/` wird das User-Interface in `App.tsx` realisiert, wie in Abschnitt 4.3 begründet, wobei je nach Pipeline-Zustand zwischen `IdleView`, `DownloadingView` und der Hauptansicht mit Aufnahmesteuerung gewechselt wird, wie in Tabelle 4 gefordert.

5.5.1 Statusanzeige und Aufnahmesteuerung

Weil das Popup beim Schließen seinen React-State verliert, wird der Pipeline-Status in `usePipelineStatus` beim Mounten per `GET_STATUS` und fortlaufend über `STATUS_UPDATE` rekonstruiert, wie in Zeile 7 des Codeausschnitts 5.17 [Google, 2024d]. Über `role="status"` und `aria-live="polite"` wird der Status von Screenreadern vorgelesen, wie in Tabelle 4 gefordert [Mozilla, 2024c].

```

1  useEffect(() => {
2    const listener = (msg: unknown) => {
3      if (
4        typeof msg === 'object' &&
5        msg !== null &&
6        'type' in msg &&
7        (msg as Record<string, unknown>).type === 'STATUS_UPDATE'
8      ) {
9        const update = msg as StatusUpdateMessage;
10       setState(update.state);
11       setError(update.error ?? null);
12       setWarning(update.warning ?? null);
13       setStep(update.step);
14       if (update.transcript !== undefined) setTranscript(update.transcript);
15       if (update.plannedActions !== undefined)
16         setPlannedActions(update.plannedActions);
17       if (update.actionExplanation !== undefined)
18         setActionExplanation(update.actionExplanation);
19       if (update.loaded !== undefined && update.total !== undefined) {
20         setDownloadProgress({ loaded: update.loaded, total: update.total });
21       }
22     }
23   };
24   browser.runtime.onMessage.addListener(listener);

```

Codeausschnitt 5.17: Nachrichtenempfang in `entrypoints/popup/hooks/usePipelineStatus.ts`

Über den Aufnahme-Button werden `START_RECORDING`- oder `STOP_RECORDING`-Nachrichten gesendet, wobei er über `disabled` gesperrt und über `aria-label` sowie `aria-pressed` beschriftet wird, wie in Tabelle 7 gefordert, und bei laufender Verarbeitung wird über die Escape-Taste eine `CANCEL_PIPELINE`-Nachricht gesendet, deren Verarbeitung in Abschnitt 5.6 beschrieben wird [World Wide Web Consortium (W3C), 2023c].

5.5.2 Aktionsprotokoll und Erfolgsfeedback

Sobald ein Aktionsplan ausgeführt wird, werden zusätzlich zum Status die Aktionen und eine LLM-generierte Erklärung übertragen, wie in Zeile 2 des Codeausschnitts 5.18. Die `plannedActions` werden als nummerierte Liste über `ActionRow` aus Zeile 11 des Codeausschnitts 5.19 dargestellt, womit ein Protokoll angezeigt wird, wie in Tabelle 4 gefordert. Die `actionExplanation` wird unterhalb der Aktionsliste kursiv eingeblendet, sobald der Zustand auf `ready` wechselt, wodurch ein Erfolgsfeedback gegeben wird, wie in Tabelle 5 gefordert. Schlägt eine Aktion fehl, wird das über `summarizeFailedActions()` gebildete Fehlerfeld befüllt, dessen Anzeige in Abschnitt 5.6 beschrieben wird.

```
1 await transitionTo('ready');
2 await broadcastStatus({
3   state: 'ready',
4   transcript: getTranscript(),
5   plannedActions: getPlannedActions(),
6   actionExplanation: getActionExplanation(),
7   executionResult: execResponse,
8   error: failureSummary,
9 });
```

Codeausschnitt 5.18: Statusrückmeldung nach Ausführung in `entrypoints/background/handlers/action-planning.ts`

```
1 import { DOMAction } from '@entrypoints/background/types';
2
3 export default function ActionRow({
4   action,
5   index,
6 }: {
7   action: DOMAction;
8   index: number;
9 }) {
10   return (
11     <li className="action-row">
12       <span className="action-num">{index + 1}</span>
13       <span className="action-type">{action.type}</span>
14       <code className="action-target">{action.selector}</code>
15       {action.value && <span className="action-value">{action.value}</span>}
16     </li>
17   );
18 }
```

Codeausschnitt 5.19: Darstellung einer Aktion in `entrypoints/popup/components/ActionRow.tsx`

5.5.3 Telemetrie-Export

Ergänzt wird das Popup um einen Telemetrie-Bereich, über den Messwerte freiwillig exportiert werden, wie der in Kapitel 6 beschriebene Nutzertest voraussetzt. Aktiviert wird die Erfassung der in `utils/telemetry.ts` definierten Kennzahlen wie `pipeline_total`, `replanIterations` und `actionCount` über die Checkbox „Enable telemetry“, während eine zweite Checkbox „Include transcripts in export“ steuert, ob das identifizierende Sprachtranskript nur bei ausdrücklicher Zustimmung exportiert wird, wie die Einwilligungserklärung im Testprotokoll aus Anhang 7.1 vorsieht.

Ausgelöst wird der Export über einen Button, der die Metriken als anonymisierte JSON-Datei ohne Serverübertragung speichert, wie es NFA-1 aus Tabelle 6 vorschreibt.

5.5.4 Barrierefreie Gestaltung

Die Farbpalette in `App.css` wird für hohe Kontraste ausgewählt, wie in Zeile 5 des Codeausschnitts 5.20, womit nach der in WCAG 2.2 normierten Formel aus relativer Leuchtdichte Verhältnisse zwischen rund 4,7 zu 1 und 15,4 zu 1 erreicht werden, über dem WCAG-AA-Schwellenwert von 4,5 zu 1 [World Wide Web Consortium (W3C), 2023a] [World Wide Web Consortium (W3C), 2023d]. Für assistive Technologien werden native, tastaturbedienbare Elemente wie `button` und `input` verwendet, womit Switch Access und virtuelle Tastaturen unterstützt werden [World Wide Web Consortium (W3C), 2023c]. Neu erscheinende Bereiche werden über `transcriptRef` und `actionsRef` fokussiert, Animationen über `prefers-reduced-motion` deaktiviert [World Wide Web Consortium (W3C), 2023b]. In `IdleView` werden beide Modelle samt Größe und Download-Reihenfolge gelistet, wodurch der sequenzielle Download nachvollziehbar wird.

```
1 :root {
2   --bg: #ffffff;
3   --surface: #f8f9fa;
4   --border: #212529;
5   --text: #212529;
6   --text-sub: #495057;
7   --text-muted: #6c757d;
8   --accent: #0043ce;
9   --accent-hover: #0036a3;
10  --accent-text: #ffffff;
11  --error-bg: #fff5f5;
12  --error-text: #c92a2a;
13  --success-bg: #f8fff8;
14  --radius: 4px;
15 }
```

Codeausschnitt 5.20: Farbvariablen für hohe Kontraste in `entrypoints/popup/App.css`

5.6 Fehlerbehandlung

Die Fehlerbehandlung wird nach dem Zuverlässigkeitsprinzip der Aktionsplanung aus Abschnitt 5.3 geführt, wonach jeder Fehler an der frühesten Übergangsstelle abgefangen, wo möglich wiederhergestellt und andernfalls als verständliche Meldung mitgeteilt wird, sodass stets in einen definierten Zustand übergegangen und nie blockiert wird. Weil Audio-Transkription, Modellplanung und Browseraktion jeweils eigene Fehlerquellen sind, wird an jeder dieser Quellen ein Hinweis gesendet und, soweit wiederherstellbar, eine alternative Strategie versucht, womit die in Tabelle 5 geforderten Meldungen bei Audio- und Planungsfehlern sowie der Fehlerstatus mit Ersatzstrategie bei Aktionsfehlern abgedeckt werden. An jeder Pipeline-Grenze wird deshalb validiert, vom Transkript-Gate `isTranscriptActionable()` vor der Planung über die Selektor- und Typenprüfung vor der Ausführung bis zur in Abschnitt 5.4 eingeführten Fehlerkategorie, anhand derer datengetrieben über Replan oder Abbruch entschieden wird. Neben der in Abschnitt 5.3.5 beschriebenen Verwaltungssicherung wird auch ein Nutzerabbruch behandelt, über den laufende Inferenzen vorzeitig beendet und asynchrone Arbeiten invalidiert werden, wonach in den Zustand `ready` übergegangen wird. Vorübergehende Fehler werden nicht weitergereicht, sondern durch Wiederholung absorbiert, sodass die Content-Script-Injektion verzögert erneut versucht und ein erkannter Berechtigungsfehler in den wiederverwendbaren Berechtigungs-Tab aus Abschnitt 5.1.2 umgeleitet wird [Google, 2024d].

Im folgenden Kapitel 6 werden die Ergebnisse der zuvor beschriebenen Implementierung an Webseiten des alltäglichen Gebrauchs evaluiert und durch einen Nutzertest mit einer sehbehinderten Person überprüft. Dabei wird ebenfalls diskutiert, ob und inwiefern die Forschungsfrage damit beantwortet werden kann, ergänzt durch Betrachtungen zu den Zielerfüllungen dieser Arbeit.

6 Evaluation und Diskussion

In diesem Kapitel wird empirisch geprüft, ob die in Kapitel 5 realisierte Browser-Erweiterung die in Abschnitt 3.2 formulierten Anforderungen erfüllt, wodurch die in Abschnitt 3.1.4 gestellte Forschungsfrage abschließend eingeordnet werden kann. Dafür wird ein Nutzertest mit begleitender Telemetrie kombiniert, weil sich qualitative Beobachtung samt Interview und quantitative Kennzahlen ergänzen. Im Anschluss wird in Abschnitt 6.2 die Rückbindung an die Anforderungen vorgenommen, bevor die Grenzen dieser Untersuchung offengelegt werden.

6.1 Testszenarien und Ergebnisse

In Abschnitt 6.1.1 wird der Testaufbau erläutert. Danach werden in Abschnitt 6.1.2 die Ergebnisse der gestellten Aufgaben ausgewertet. Um anhand genauer Werte die Ergebnisse einordnen zu können, werden in Abschnitt 6.1.3 die Messwerte dargestellt. Abschließend wird das Interview mit der Testperson für die Darstellung qualitativen Feedbacks beleuchtet.

6.1.1 Testaufbau

Rekrutiert wird eine einzige Testperson, die gemäß Einwilligungserklärung als TP1 geführt wird und bei der eine Netzhautdegeneration mit Nutzung einer Bildschirmlupe zwischen 500 und 800 Prozent sowie einem ergänzenden Screenreader vorliegt. Telemetriedaten werden als quantitative Ergänzung zur Beobachtung herangezogen, wodurch keine statistisch belastbare Aussage, sondern eine Plausibilisierung anhand der in NFA-6 aus Tabelle 7 adressierten Zielgruppe angestrebt wird. Legitimiert wird der schmale Stichprobenumfang durch ein formatives statt summatives Vorgehen, bei dem schon kleine Stichproben die gravierendsten Bedienprobleme freilegen [Nielsen, 2000]. Hinzu tritt ein Rekrutierungsengpass, weil Personen mit einer derart spezifischen Beeinträchtigung keine breit verfügbare Stichprobe bilden und eine größere Ausschreibung den Zeitrahmen überschritten hätte, weshalb ein vertiefter Einzelfall dem oberflächlichen Sammeln mehrerer Meinungen vorgezogen wird. Kompensiert wird die geringe Fallzahl durch eine Triangulation aus Beobachtung, Interview und Telemetrie über vier Aufgaben, während die fehlende Generalisierbarkeit in Abschnitt 6.2.3 aufgegriffen wird. Durchgeführt wird der Test im Büro auf einem Notebook mit Windows 11, Intel-Core-i5-1245U-Prozessor und 16 GB Arbeitsspeicher, ergänzt um einen externen 21:9-Monitor sowie Google Chrome 150.0.7871.125, wobei als Eingabemethode der Hotkey gewählt wird.

Übernommen wird für die Bewertung eine dreistufige Rubrik aus dem Testprotokoll, nach der eine Aufgabe als erfolgreich bei höchstens einem Versuch ohne Eingriff und ohne Falschaktion, als teilweise bei mindestens zwei Versuchen oder manueller Hilfe und als fehlgeschlagen beim Ausbleiben des Endzustands oder bei Falschaktion eingestuft wird. Vier realitätsnahe Aufgaben werden TP1 gesprochen vorgegeben, wie in Tabelle 11 zusammengefasst wird.

Nr.	Szenario	Gesprochener Befehl
1	Interne Arbeitszeiterfassung	„Trage im ersten Textfeld 8 Uhr zwölf ein.“
2	Google-Websuche	„Gib in das Suchfeld ein, Borussia Dortmund, und führe die Suche durch.“
3	Anmeldeformular (Confluence)	„Trage als Benutzername P00 und als Passwort 12345 und melde dich an.“
4	Amazon-Produktsuche	„Trage in die Suchleiste Samsung Galaxy S26 ein und starte die Suche.“

Tabelle 11: Testaufgaben mit gesprochenem Befehl

6.1.2 Aufgabenergebnisse

Ausgewertet werden die vier Aufgaben anhand der in Abschnitt 6.1.1 definierten Rubrik, wie Tabelle 12 verdichtet darstellt. Lediglich die Websuche wird erfolgreich abgeschlossen, während die Zeiterfassung an einer Endlosschleife scheitert, die durch wiederholte Generierung eines nicht existierenden Selektors ausgelöst wird und laut Beobachtungsnotiz das Aktionsmodell in einen Panikzustand versetzt. Beim Anmeldeformular wird im zweiten Versuch anstelle des Login-Buttons die Suchfunktion geöffnet, weshalb die Aufgabe trotz plausibler Dateneingabe nur teilweise gelingt, zumal die Ziffernfolge des Passworts als ausgeschriebene Wörter statt als Zahlen transkribiert wird. Bei der Amazon-Aufgabe wird anstelle des Such-Buttons ein zufälliges Produkt der Startseite angeklickt, weshalb ein zweiter Versuch nötig wird, um das Ergebnis zu retten. Als Hauptfehlerquelle kristallisiert sich die Selektor- und Element-Disambiguierung heraus, wie sie in Abschnitt 5.4 beschrieben wird, wohingegen die Spracherkennung außer bei Zahlwörtern zuverlässig arbeitet.

Aufg.	Ergebnis	Versuche	Eingriff	Falschaktion
1	fehlgeschlagen	3	nein	nein (Endlosschleife)
2	erfolgreich	1	nein	ja (Klick auf „Bilder“)
3	teilweise	3	nein	ja (Suche statt Login)
4	teilweise	2	nein	ja (zufälliges Produkt)

Tabelle 12: Verdichtete Aufgabenergebnisse

6.1.3 Telemetrie

Herangezogen wird neben der Beobachtung für jede Aufgabe die über den in Abschnitt 5.5.3 beschriebenen Export erzeugte Telemetrie-Datei, aus der die Dauer `pipeline_total`, die Anzahl `replanIterations` sowie ein etwaiges Auslösen des deterministischen Fallbacks entnommen werden. Zwischen 13,4 Sekunden für das bloße Öffnen eines Suchergebnisses und 78,3 Sekunden für den zweiten Amazon-Versuch schwankt die gemessene Dauer, womit die Latenzgrenze von zehn Sekunden gemäß NFA-11 aus Tabelle 9 in keinem Fall eingehalten wird. Die `replanIterations` verharren meist bei eins und steigen vereinzelt auf zwei, wodurch die in Abschnitt 5.3.5 beschriebene Replan-Schleife greift, die Gesamtdauer jedoch zusätzlich verlängert. Ausgelöst wird der deterministische Fallback aus Abschnitt 5.3.6 in keinem der vier Testfälle, woraus hervorgeht, dass das Modell stets einen Plan liefert und die Fehler in der Planqualität und der Selektorauflösung liegen, nicht im Fallback-Pfad. Auffällig weit auseinander liegen die subjektiv geschätzte und die gemessene Wartezeit, wie Tabelle 13 zeigt, da TP1 die Dauer durchgängig auf ungefähr zehn Sekunden schätzt, obwohl bis zum Achtfachen gemessen wird, was in Abschnitt 6.1.4 wieder aufgegriffen wird.

Aufg.	Subjektiv geschätzt	Gemessen (<code>pipeline_total</code>)
1	ca. 10 s	74,4 s / 72,2 s
2	nicht beziffert	70,4 s / 13,4 s
3	ca. 10 s je Befehl	17,2 s bis 20,7 s
4	über 10 s	73,8 s / 78,3 s

Tabelle 13: Subjektiv geschätzte vs. gemessene Wartezeit je Aufgabe

6.1.4 Qualitatives Feedback

Im Abschlussinterview auf einer fünfstufigen Skala wird die allgemeine Bedienbarkeit mit 4 und die Verständlichkeit des visuellen Feedbacks mit 5 bewertet, wie Tabelle 14 auflistet, während die Erleichterung gegenüber der gewohnten Bedienung nur mit 3 und das Vertrauen in die ausgeführten Aktionen mit lediglich 2 eingestuft wird. Wörtlich hält TP1 fest, dass die Dauer aktuell zu lang sei, weil die Aktionsplanung zu viel Zeit beanspruche, ergänzt jedoch, die Extension werde gern genutzt, sobald die Qualität ausreiche. Positiv hervorgehoben wird zudem die Darstellung des Transkripts als Rückmeldung sowie die Sichtbarkeit des Status über die Farbgebung. Weder durch die Lupennutzung noch durch den geringen Bildschirmabstand wird laut Beobachtungsnotiz eine zusätzliche Erschwernis festgestellt, wohingegen ein struktureller Vorteil der Sprachsteuerung gegenüber Maus und Tastatur wegen der hohen Latenz ausbleibt und erst bei einer Ausführungszeit von rund fünf Sekunden zu erwarten wäre, was als Grenze der Extension statt des Testdesigns in Abschnitt 6.2.3 eingeordnet wird. Für die vollständigen Rohdaten samt Einwilligung und Teilkriterien je Aufgabe wird auf das Testprotokoll in Anhang 7.1 verwiesen.

Frage	Wert (1 bis 5)
Bedienbarkeit der Extension	4
Verständlichkeit des visuellen Feedbacks	5
Erleichterung gegenüber gewohnter Bedienung	3
Vertrauenswürdigkeit der ausgeführten Aktionen	2

Tabelle 14: Abschlussinterview mit TP1

6.2 Zielerfüllung und Limitierungen

Rückgebunden werden die Testergebnisse aus Abschnitt 6.1 an den Anforderungskatalog aus Abschnitt 3.2, getrennt in erfüllte und offene Anforderungen, bevor die Limitierungen benannt werden.

6.2.1 Erfüllte Anforderungen

Aus Tabelle 1 werden FA-01, FA-02, FA-04 und FA-05 erfüllt, aus Tabelle 2 FA-06 bis FA-09, aus Tabelle 3 FA-11, FA-13 und FA-15, aus Tabelle 4 FA-16 bis FA-18 sowie aus Tabelle 5 FA-19, FA-20 und FA-22, jeweils durch Aufnahme, Transkription, Planung, Ausführung, Fehlermeldung und Anzeige, wobei FA-16, FA-17 und FA-22 durch den Interviewwert 5 aus 5 laut Abschnitt 6.1.4 bestätigt werden. Weil die Verarbeitung vollständig auf dem Gerät erfolgt und nach dem initialen Modell-Download keine Verarbeitungsdaten an einen Server übertragen werden, werden NFA-1 und NFA-2 aus Tabelle 6 sowie NFA-10, NFA-13 und NFA-14 aus Tabelle 9 erfüllt, und zwar durch eine Ladezeit deutlich unter 30 Sekunden, rund eine Sekunde Transkriptionswartzeit und ausbleibende erneute Modell-Downloads. NFA-3 bis NFA-6 aus Tabelle 7 und NFA-16 aus Tabelle 10 werden durch die barrierefreie, neben Screenreader und Lupe lauffähige Oberfläche erfüllt, und NFA-7 sowie NFA-8 aus Tabelle 8 durch die Lauffähigkeit unter Chrome und Firefox.

6.2.2 Nicht bzw. teilweise erfüllte Anforderungen

Klar verfehlt wird NFA-11 aus Tabelle 9, weil laut Abschnitt 6.1.3 bis zu 78,3 statt der geforderten zehn Sekunden gemessen werden, und knapp verfehlt wird NFA-12 aus derselben Tabelle, weil die Speichernutzung mit rund 3 GB die geforderten 2 GB übersteigt, eventuell bedingt durch andere Laptop-Prozesse. Bei zwei der vier Aufgaben tritt eine Falschaktion auf, wodurch FA-12 aus Tabelle 3 zur korrekten Element-Identifikation, konkret der Login-Klick und die Produktauswahl anstelle des Such-Buttons, als nicht durchgängig erfüllt gilt. Teilweise erfüllt bleibt FA-10 aus Tabelle 2, weil die Replan-Schleife aus Abschnitt 5.3.5 zwar auf fehlerhafte Pläne reagiert, aber nicht immer einen brauchbaren alternativen Plan liefert.

Nicht erfüllt wird FA-21 aus Tabelle 5 in der Zeiterfassungsaufgabe, weil bei der wiederholt fehl-schlagenden Aktion weder ein Fehlerstatus angezeigt noch eine alternative Strategie versucht wird, sondern eine Endlosschleife entsteht. Weil die Extension über keine Sprachausgabe verfügt und TP1 nach dem Briefinghinweis während laufender Verarbeitung nicht erneut aufnimmt, bleibt TP1 ohne Hinweis auf den Stillstand, weshalb dieser Fall als kritisch eingestuft wird. Offenbar wird eine Schwäche bei FA-03 aus Tabelle 1 zur korrekten deutschen Transkription bei Zahlen, da Ziffern in der Anmeldeaufgabe als ausgeschriebene Wörter erkannt werden und dadurch einen Folgefehler in der Planung nach sich ziehen.

6.2.3 Limitierungen

Technisch fällt zunächst das Fehlen einer automatischen Beendigung der Aufnahme nach Sprechende auf, ein von TP1 selbst vorgeschlagener Verbesserungspunkt. Fehleranfällig bleibt außerdem die Selektor- und Element-Vereinbarung bei dynamischen oder mehrdeutigen Seiten wie der Amazon-Produktliste und dem Confluence-Login, wobei ein Regressionstest nach TP1 zeigt, dass eine dynamisch generierte Grammatik die Halluzination nicht existierender Selektoren zuverlässig verhindert, während eine semantische Fehlwahl unter mehreren validen Kandidaten, etwa ein Klick auf den Sprachauswahl-Link „DE“ statt auf den Such-Button, als Kapazitätsgrenze des 1,5-Milliarden-Parameter-Modells bestehen bleibt. Die Aktionsplanungsdauer der Modellinferenz dominiert überdies die Gesamtlatenz, was den bereits in der Anforderungsanalyse skizzierten Zielkonflikt zwischen Leistungsfähigkeit und Nutzbarkeit bei vollständiger DOM-Übergabe bestätigt. Ein rein akustisches Feedback fehlt zudem als Rückkanal, was gerade für die adressierte Zielgruppe mit Sehbeeinträchtigung relevant ist.

Methodisch begrenzt vor allem die Fallzahl von eins ohne Vergleichsgruppe ohne Sehbeeinträchtigung die Aussagekraft des Tests. Da die gemessene Latenz die manuelle Bedienung nicht übertrifft, bleibt ein struktureller Vorteil der Sprachsteuerung gegenüber Maus und Tastatur im Test aus. Auf Browserseite wird unter Chrome und Firefox getestet, wobei beide lauffähig sind, fast alle Aufgaben aber nur unter Chrome durchgeführt werden, sodass Safari nach NFA-9 aus Tabelle 8 unberücksichtigt bleibt, ebenso wie die Zustandserhaltung nach NFA-15 aus Tabelle 10, die nicht erfasst wird, und die Scroll-Aktion nach FA-14 aus Tabelle 3, die aus Zeitgründen nicht umgesetzt wird.

Im anschließenden Kapitel 7 wird ein Fazit zur Forschungsfrage gezogen und ein Ausblick auf das Projekt gegeben.

7 Fazit

Wörtlich lautet die eingangs gestellte Forschungsfrage, wie ein technisches Hilfsmittel entworfen werden kann, sodass Menschen mit motorischer oder Sehbehinderung Webseiten ausschließlich durch Spracheingabe bedienen können, unter der Bedingung, dass verwendete KI-Modelle lokal auf dem Endgerät des Nutzers ausgeführt werden. Diese Frage wird grundsätzlich bejaht, weil eine vollständig lokale Ausführungskette aus Spracherkennung, der Aktionsplanung und der DOM-Interaktion funktionsfähig demonstriert wird und die Machbarkeit ohne die Nutzung von Cloud-Modellen bewiesen wird. Einzuschränken ist diese Antwort bei der Praxistauglichkeit, da in der Evaluation in Kapitel 6 nur eine von vier Aufgaben als erfolgreich bewertet wird und als Fehlerquellen Probleme bei der DOM-Selektor-Auswahl sowie den Anforderungen nicht entsprechende Latenzen identifiziert werden. Deswegen werden die Antwortzeit und Ausführungsgenauigkeit als noch nicht auf dem gewünschten Niveau betrachtet, das eine selbstständige Bedienung ohne Rückgriff auf Maus oder Tastatur ermöglicht und im Interviewwert von 2 aus 5 zur Vertrauenswürdigkeit widerspiegelt wird. Damit wird die Antwort auf die Forschungsfrage als im Prototypenstadium formuliert und nicht als final gelöstes Problem. Im Rückblick auf die Testergebnisse wird es als nachvollziehbar betrachtet, dass in bisher betrachteten Lösungen aus Abschnitt 3.1 größtenteils Cloud-KI-Modelle und größere Modelle genutzt werden, da durch diese mit größerem Kontext und mehr Rechenleistung bessere Ergebnisse und Antwortzeit erzielt werden können. Verfolgte Datenschutz und Offline-Vorteile der Browser-Extension werden folglich aktuell als Kompromiss mit der geringeren Leistungsfähigkeit kleiner KI-Modelle auf leistungsschwächeren Endgeräten betrachtet. Die entwickelte Software wird nach der Fertigstellung dieser Arbeit weiterentwickelt als Open-Source-Projekt auf der Plattform GitHub, damit bestehende Probleme und nicht erfüllte Anforderungen in der Zukunft behoben und erfüllt werden. So wird zum Beispiel versucht, ein automatisches Beenden der Aufnahme nach dem Absenken der Stimme, die Verbesserung der Aktionsplanung und der DOM-Selektor-Auswahl des lokalen KI-Modells, eine verbesserte Transkription von Zahlen sowie das Erreichen der Anforderung NFA-11 für eine geringe Wartezeit und bessere Performance umzusetzen. Über die Konvertierung zu einem Open-Source-Projekt werden gegenüber der in dieser Arbeit durchgeführten Evaluierung Tests mit einer größeren Teilnehmeranzahl für mehr Feedback von Menschen der Zielgruppe ermöglicht.

Auch wenn im Rahmen dieser Arbeit nicht alle Anforderungen erreicht werden, so wird trotzdem die Machbarkeit einer ausschließlich lokal ausgeführten und barrierearmen Sprachsteuerung von Webseiten gezeigt, wodurch ein Beitrag zur digitalen Teilhabe der Zielgruppe bei gewährtem Datenschutz geleistet wird.

Literaturverzeichnis

- [Browser Use Team, 2024] Browser Use Team (2024). Browser use: Make websites accessible for AI agents. Zuletzt abgerufen: 2026-07-05.
- [Cambre et al., 2021] Cambre, J., Williams, A. C., Razi, A., Bicking, I., Wallin, A., Tsai, J., Kulkarni, C., und Kaye, J. (2021). Firefox Voice: An open and extensible voice assistant built upon the web. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*, pages 1–18, New York, NY, USA. Association for Computing Machinery.
- [Cossovich et al., 2023] Cossovich, R., Hodges, S., Kang, J., und Girouard, A. (2023). Co-designing new keyboard and mouse solutions with people living with motor impairments. *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '23)*, pages 1–7. Association for Computing Machinery.
- [Deng et al., 2023] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., und Su, Y. (2023). Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36, pages 28091–28114. Curran Associates, Inc.
- [Europäisches Parlament und Rat der Europäischen Union, 2016a] Europäisches Parlament und Rat der Europäischen Union (2016a). Richtlinie (EU) 2016/2102 des europäischen parlaments und des rates über den barrierefreien zugang zu den websites und mobilen anwendungen öffentlicher stellen. Richtlinie 2016/2102, Europäische Union. Published: Amtsblatt der Europäischen Union L 327, S. 1–15.
- [Europäisches Parlament und Rat der Europäischen Union, 2016b] Europäisches Parlament und Rat der Europäischen Union (2016b). Verordnung (EU) 2016/679 des europäischen parlaments und des rates zum schutz natürlicher personen bei der verarbeitung personenbezogener daten, zum freien datenverkehr und zur aufhebung der richtlinie 95/46/EG (datenschutzgrundverordnung). Verordnung 2016/679, Europäische Union. Published: Amtsblatt der Europäischen Union L 119, S. 1–88.
- [GBD 2019 Blindness and Vision Impairment Collaborators, 2021] GBD 2019 Blindness and Vision Impairment Collaborators (2021). Causes of blindness and vision impairment in 2020 and trends over 30 years, and prevalence of avoidable blindness in relation to VISION 2020: The right to sight. *The Lancet Global Health*, 9(2):e144–e160.
- [ggml-org, 2024] ggml-org (2024). GGUF Specification. GitHub. Abgerufen am 30. Juni 2026.
- [Google, 2024a] Google (2024a). Chrome extensions: Offscreen documents.
- [Google, 2024b] Google (2024b). chrome.commands.
- [Google, 2024c] Google (2024c). Manifest - content security policy.
- [Google, 2024d] Google (2024d). Message passing.

- [He et al., 2024] He, H., Yao, W., Ma, K., Yu, W., Dai, Y., Zhang, H., Lan, Z., und Yu, D. (2024). WebVoyager: Building an end-to-end web agent with large multimodal models. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Volume 1: Long Papers*, pages 6864–6890. Association for Computational Linguistics.
- [Huang et al., 2024] Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., und Chen, E. (2024). Understanding the Planning of LLM Agents: A Survey.
- [Hugging Face, 2024a] Hugging Face (2024a). GGUF. Hugging Face Documentation. Abgerufen am 30. Juni 2026.
- [Hugging Face, 2024b] Hugging Face (2024b). Transformers.js: Run hugging face transformers in the browser.
- [Hwang et al., 2026] Hwang, J., Zhang, X., und Padwal, V. (2026). WebChallenger: A reliable and efficient generalist web agent.
- [Lai et al., 2024] Lai, H., Liu, X., long, I. L., Yao, S., Chen, Y., Shen, P., Yu, H., Zhang, H., Zhang, X., Dong, Y., und Tang, J. (2024). AutoWebGLM: A large language model-based web navigating agent. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, pages 5295–5306. Association for Computing Machinery.
- [Microsoft, 2024] Microsoft (2024). ONNX Runtime Web Documentation.
- [Microsoft Corporation, 2024] Microsoft Corporation (2024). Generics – TypeScript Handbook. Dokumentation. Abgerufen am 30. Juli 2026.
- [Mozilla, 2024a] Mozilla (2024a). AbortController. MDN Web Docs. Abgerufen am 30. Juli 2026.
- [Mozilla, 2024b] Mozilla (2024b). Anatomy of an extension. MDN Web Docs. Abgerufen am 30. Juni 2026.
- [Mozilla, 2024c] Mozilla (2024c). ARIA live regions. MDN Web Docs. Abgerufen am 14. Juli 2026.
- [Mozilla, 2024d] Mozilla (2024d). CSS selectors. MDN Web Docs. Abgerufen am 30. Juli 2026.
- [Mozilla, 2024e] Mozilla (2024e). JSON.parse(). MDN Web Docs. Abgerufen am 30. Juli 2026.
- [Mozilla, 2024f] Mozilla (2024f). MediaDevices: getUserMedia() method. MDN Web Docs. Abgerufen am 30. Juli 2026.
- [Mozilla, 2024g] Mozilla (2024g). MediaRecorder. MDN Web Docs. Abgerufen am 30. Juli 2026.
- [Mozilla, 2024h] Mozilla (2024h). MutationObserver. MDN Web Docs. Abgerufen am 30. Juli 2026.
- [Mozilla, 2024i] Mozilla (2024i). Object.getPrototypeOf(). MDN Web Docs. Abgerufen am 30. Juli 2026.

- [Mozilla, 2024j] Mozilla (2024j). What are extensions? MDN Web Docs. Abgerufen am 30. Juni 2026.
- [ngxson, 2024] ngxson (2024). wllama: WebAssembly Binding for llama.cpp. GitHub. GitHub-Repository, zuletzt abgerufen am 30. Juni 2026.
- [Nielsen, 2000] Nielsen, J. (2000). Why You Only Need to Test with 5 Users. Nielsen Norman Group. Abgerufen am 23. Juli 2026.
- [NVIDIA, 2024] NVIDIA (2024). Parakeet TDT 0.6b v3: Multilingual ASR model card.
- [Pantelaïos and Kapravelos, 2024] Pantelaïos, N. und Kapravelos, A. (2024). Manifest v3 unveiled: Navigating the new era of browser extensions.
- [Prabhavalkar et al., 2023] Prabhavalkar, R. et al. (2023). End-to-End Speech Recognition: A Survey. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*.
- [Prabhavalkar et al., 2024] Prabhavalkar, R., Hori, T., Sainath, T. N., Schlüter, R., und Watanabe, S. (2024). End-to-end speech recognition: A survey. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:325–351.
- [Pucci et al., 2024] Pucci, E., Piro, L., Possaghi, I., Mulfari, D., und Matera, M. (2024). Co-designing the integration of voice-based conversational AI and web augmentation to amplify web inclusivity. *Scientific Reports*, 14:16162.
- [Qi et al., 2024] Qi, Z., Liu, X., long, I. L., Lai, H., Sun, X., Zhao, W., Yang, Y., Yang, X., Sun, J., Yao, S., Zhang, T., Xu, W., Tang, J., und Dong, Y. (2024). WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning.
- [Rabin and Scott, 1959] Rabin, M. O. und Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2):114–125.
- [Ruan and others, 2024] Ruan, C. F. und others (2024). WebLLM: A high-performance in-browser LLM inference engine.
- [Shen et al., 2024] Shen, A., Lai, Z., und Li, D. (2024). Exploring quantization techniques for large-scale language models: Methods, challenges and future directions. *Proceedings of the 2024 9th International Conference on Cyber Security and Information Engineering (ICCSIE '24)*, pages 783–790. Association for Computing Machinery.
- [Sousa and Kern, 2023] Sousa, S. und Kern, R. (2023). How to keep text private? a systematic review of deep learning methods for privacy-preserving natural language processing.
- [Wang et al., 2024] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., und Wen, J.-R. (2024). A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*.
- [WebAIM, 2024a] WebAIM (2024a). Screen reader user survey #10 results. Technischer bericht, WebAIM (Web Accessibility In Mind).

- [WebAIM, 2024b] WebAIM (2024b). The WebAIM million: The 2024 report on the accessibility of the top 1,000,000 home pages. Technischer bericht, WebAIM (Web Accessibility In Mind).
- [Wei et al., 2022] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., und Fedus, W. (2022). Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research*.
- [World Health Organization, 2022] World Health Organization (2022). *Global Report on Health Equity for Persons with Disabilities*. World Health Organization.
- [World Wide Web Consortium (W3C), 2023a] World Wide Web Consortium (W3C) (2023a). Understanding Success Criterion 1.4.3: Contrast (Minimum). W3C WAI. Abgerufen am 14. Juli 2026.
- [World Wide Web Consortium (W3C), 2023b] World Wide Web Consortium (W3C) (2023b). Understanding Success Criterion 2.3.3: Animation from Interactions. W3C WAI. Abgerufen am 14. Juli 2026.
- [World Wide Web Consortium (W3C), 2023c] World Wide Web Consortium (W3C) (2023c). Using ARIA. W3C Working Group Note. Abgerufen am 14. Juli 2026.
- [World Wide Web Consortium (W3C), 2023d] World Wide Web Consortium (W3C) (2023d). Web content accessibility guidelines (WCAG) 2.2. W3c recommendation, World Wide Web Consortium.
- [WXT Project, 2024a] WXT Project (2024a). Entrypoints. Dokumentation. Abgerufen am 13. Juli 2026.
- [WXT Project, 2024b] WXT Project (2024b). Manifest. Dokumentation. Abgerufen am 13. Juli 2026.
- [WXT Project, 2024c] WXT Project (2024c). WXT: Next Gen Web Extension Framework. Dokumentation. Abgerufen am 06. Juli 2026.
- [Yao et al., 2023] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., und Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *11th International Conference on Learning Representations (ICLR 2023)*.
- [ysdede, 2025] ysdede (2025). parakeet.js: NVIDIA Parakeet Speech Recognition for the Browser (WebGPU/WASM) Powered by ONNX Runtime Web. GitHub. GitHub-Repository, zuletzt abgerufen am 14. Juli 2026.
- [Yu et al., 2025] Yu, Y., Ryskeldiev, B., Tsutsui, A., Gillingham, M., und Wang, Y. (2025). From cluttered to clear: Improving the web accessibility design for screen reader users in e-commerce with generative AI. *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '25)*, pages 1–19. Association for Computing Machinery.

- [Zerhoudi and Granitzer, 2026] Zerhoudi, S. und Granitzer, M. (2026). In-browser agents for search assistance. *Proceedings of the 2026 ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR '26)*, pages 345–349, Seattle, WA, USA. Association for Computing Machinery.
- [Zhang et al., 2025] Zhang, Y., Liu, M., Wang, H., Ma, Y., Huang, G., und Liu, X. (2025). Research on WebAssembly runtimes: A survey. *ACM Transactions on Software Engineering and Methodology*, 34(8):1–47.
- [Zhao et al., 2024] Zhao, W. X. et al. (2024). A Survey of Large Language Models. *arXiv preprint arXiv:2303.18223*.
- [Zheng et al., 2024] Zheng, B., Gou, B., Kil, J., Sun, H., und Su, Y. (2024). GPT-4V(ision) is a generalist web agent, if grounded.
- [Zheng et al., 2025] Zheng, Y., Chen, Y., Qian, B., Shi, X., Shu, Y., und Chen, J. (2025). A review on edge large language models: Design, execution, and applications. *ACM Computing Surveys*, 57(8):209:1–209:35.

KI-Verzeichnis

KI-basiertes Hilfsmittel	Einsatzform	Prompts	Betroffene Teile der Arbeit
Claude	Formulierungshilfe, Korrekturhilfe	Schreib mir Formulierungshilfen für die Einführung, die Grundlagen und den Stand der Technik. Schlage mir präzisere und kürzere Formulierungen für den Entwurf und die Implementierung vor, ohne Informationsgehalt zu entfernen. Schlage kürzere, flüssigere Formulierungen für die Evaluation vor, die den wissenschaftlichen Stil der Arbeit einhalten. Lies die komplette Arbeit und zeige mir Stellen, die korrigiert werden müssten wegen Rechtschreibung, Grammatik oder falschem Tempus.	Kapitel 1 (Einführung), Kapitel 2 (Grundlagen), Kapitel 3.1 (Stand der Technik), Kapitel 4 (Entwurf), Kapitel 5 (Implementierung), Kapitel 6 (Evaluation)
Anara	Analyse und Extraktion von Fachliteratur	Analysiere alle Quellen aus dem Ordner Einführung (Grundlagen separat) und zeige mir mögliche relevante Stellen für meine Planung des Kapitels. Analysiere die Quellen im Ordner Stand der Technik und extrahiere die zentralen Methoden, Ergebnisse und Limitationen der jeweiligen Arbeiten mit Seitenangaben. Durchsuche die Quellen im Ordner Implementierung nach Belegen für die gewählten Technologien (wllama, ONNX Runtime Web, WXT) und nenne die passenden Stellen.	Kapitel 1 (Einführung), Kapitel 2 (Grundlagen), Kapitel 3.1 (Stand der Technik), Kapitel 5 (Implementierung)

Anhang

7.1 Testprotokoll TP1

Auf der folgenden Seite wird das vollständige und von beiden Parteien unterschriebene Testprotokoll des in Kapitel 6.1 ausgewerteten Nutzertests angehängen. Einschließend werden alle Einwilligungsangaben, alle Teilkriterien je Aufgabe und die Rohdaten der Telemetrie im Testprotokoll angegeben.

Testprotokoll für Nutzertest mit sehbehinderter Person

Protokoll für einen Nutzertest der Extension mit einer Testperson, die entweder einen sehr geringen Bildschirmabstand benötigt und/oder eine Bildschirmlupe nutzt. Ziel ist es, die Ergebnisse und das Feedback der Testperson für Evaluation und Diskussion der Thesis auswertbar festzuhalten.

Datum	21.07.2026
Ort / Setting	Im Büro
Durchführende Person	Theo Leuthardt
Testperson	Matthias Köhler
Anonymisierte Kennung für die Thesis	TP1

1. Einverständniserklärung

Dieses Dokument dient zugleich als Einverständniserklärung. Die Testperson (TP) wird vor dem Test mündlich über Ziel, Ablauf und die folgenden Punkte aufgeklärt und bestätigt die Einwilligung unten durch Unterschrift.

1.1 Zweck & Rahmen

Im Rahmen einer Bachelorarbeit wird ein Nutzertest der Browser-Extension durchgeführt. Ziel ist, die Bedienbarkeit der Extension durch eine Person mit Sehbeeinträchtigung zu beobachten und das Feedback für die Evaluation der Thesis auszuwerten. Dauer: ca. 30 Min. Abbruch jederzeit möglich, ohne Angabe von Gründen und ohne Nachteile.

1.2 Was erfasst wird und was nicht

Datenart	Wird erfasst?	Verbleib / Verwendung
Mikrofon-Audio	Nur live zur Spracherkennung auf dem Gerät	Verlässt nicht das Gerät, nicht gespeichert, nicht aufgezeichnet.
Telemetrie	Ja (Standard-Export)	Export als anonymisiertes JSON ohne Transkript, auf dem Gerät der durchführenden Person gespeichert, ausgewertet für die Thesis.
Transkript	Nur bei ausdrücklicher Einwilligung.	Wird nur exportiert, wenn die Option „Include transcripts“ im Popup aktiviert ist.
Beobachtungsnotizen & Zitate	Ja	Wörtliche Zitate werden anonymisiert als TP1 in die Thesis übernommen, also kein Klarname.
Videoaufzeichnung	Nein	–

1.3 Einwilligung

Ich willige ein in:

- ☒ Aufklärung über Ziel und Ablauf erhalten
- ☒ Live-Spracherkennung auf dem Gerät
- ☒ Export und Speicherung der anonymisierten Telemetrie als JSON
- ☒ Export der Transkripte
- ☒ Anonymisierte wörtliche Zitate in der Thesis

☒ Recht auf jederzeitigen Abbruch ohne Begründung

☒ Recht auf nachträgliche Einsicht und Widerruf

1.4 Aufbewahrung & Löschung

Telemetrie-JSONs und Notizen werden auf dem Gerät der durchführenden Person gespeichert und nach Abschluss / Abgabe der Thesis gelöscht, spätestens zum 01.09.2026. Auf Wunsch erhält TP1 vorab Einsicht in die sie betreffenden Daten.

1.5 Unterschrift

Durchführende Person

Name

Theo Leuthardt

Unterschrift

Theo
Leuthardt

Digital signiert von Theo Leuthardt
DN: C=DE, CN=Theo Leuthardt, GN=Theo, O=
Bundesdruckerei GmbH, SN=Leuthardt, ST=
Berlin, organizationIdentifier=NTRDE-
F1103R,HRB70764 B
Grund: Ich bin der Verfasser dieses Dokuments
Ort: Berlin
Datum: 2026.07.27 11:48:26+02'00'
Foxit PDF Editor Version: 2026.1.1

Datum

27.07.2026

Testperson (TP1)

Name (Klarname, nur hier)

[Mathias Köhler](#)

Unterschrift

Mathias
Köhler

Digital signiert von Mathias Köhler
DN: C=DE, CN=Mathias Köhler, GN=Mathias, L=
Berlin, O=Bundesdruckerei GmbH, SN=Köhler, ST=
Berlin, organizationIdentifier=NTRDE-
F1103R,HRB70764 B, serialNumber=CSM050653357
Grund: Ich stimme den Bedingungen zu, die durch
das Signieren dieses Dokuments durch mich definiert
werden
Ort: Berlin
Datum: 2026.07.23 10:59:59+02'00'
Foxit PDF Editor Version: 2026.1.1

Datum

[22.07.2026](#)

Ort

Berlin

2. Angaben zur Sehbeeinträchtigung

Feld	Angabe
Art der Sehbeeinträchtigung	Netzhaut Degeneration (Nähe und Ferne nicht scharf sehen können)
Nutzt Bildschirmlupe?	Ja, von Windows
Nutzt Screenreader zusätzlich?	Ja vom Browser oder Webseite
Üblicher Bildschirmabstand	5-10cm
Übliche Vergrößerungsstufe (z. B. Browser-Zoom, OS-Zoom, Lupen-Faktor)	500-800%
Sonstige assistive Technologien	Sonst nichts weiter.

3. Testumgebung

Feld	Angabe
Gerät	Windows 11 Laptop (Intel Core i5-1245U, 16GB RAM) + externer 21:9 Monitor + Tastatur + Maus
Browser + Version	Google Chrome, Version: 150.0.7871.125
Verwendete Testseite(n)	Interne Zeiterfassung, Google, Jira, Amazon
Eingesetzte Vergrößerung während des Tests	500-800%
Mikrofon / Audio-Setup	Internes Mikrofon des Laptops mit Aktivierung von Windows Voice Clarity und Sprachfokus

3.1 Briefing für TP1

Nach der Einverständniserklärung und vor dem Warmup TP1 mündlich durchgehen. Die drei Punkte sind kritisch, weil TP1 auf visuelles Feedback angewiesen ist und die Extension keine Sprachausgabe hat.

1. **Latenz ist normal**
2. **Nicht neu aufnehmen, während es denkt**
3. **Abbrechen mit ESC oder Cancel-Button**

Feld	Angabe
Briefing vollständig gegeben? (ja / nein)	ja
TP1 hat Latenz-/Warte-Regel verstanden? (ja / nein)	ja
Gewählte Eingabemethode (Popup / Hotkey)	Hotkey

4. Aufgaben

Für jede Aufgabe: gesprochener Befehl, Ergebnis und Beobachtungen separat notieren.

4.0 Erfolgskriterien-Rubrik

Um die Bewertung „erfolgreich / teilweise / fehlgeschlagen“ reproduzierbar zu machen, wird sie pro Aufgabe operationalisiert. Je Aufgabe gibt es zusätzlich konkrete, beobachtbare Teilkriterien (siehe 4.2–4.5). Die Gesamtbewertung ergibt sich aus der folgenden Regel, angewendet auf den intendierten Endzustand der Aufgabe:

Bewertung	Definition
erfolgreich	Intendierter Endzustand mit ≤ 1 Versuch erreicht, kein manueller Eingriff durch TP, keine falsche Aktion ausgeführt.
teilweise	Endzustand erst nach ≥ 2 Versuchen ODER mit manueller Hilfe erreicht, oder nur ein Teil der Aufgabe ausgeführt (z. B. Suchbegriff eingetragen, Suche aber nicht gestartet, Ergebnis also nicht geöffnet).
fehlgeschlagen	Endzustand auch nach Versuchen / manueller Hilfe nicht erreicht, oder eine falsche Aktion wurde ausgeführt.

4.1 Aufgabe 1: Interne Arbeitszeiterfassung im HR-Portal

Gesprochener Auftrag an TP: Trage in der internen Zeiterfassung deine heutige Arbeitszeit oder externe Pause ein.

Teilkriterien (intendierter Endzustand):

1. Das richtige Feld für die externe Pause wurde gefunden und befüllt.
2. Der Pausenwert ist korrekt eingetragen.
3. Der Eintrag wurde gespeichert / abgesendet (nicht nur eingetippt).

Feld	Angabe
Tatsächlich gesprochener Befehl	Trage im ersten Textfeld 8 Uhr zwölf ein.
Ergebnis (erfolgreich / teilweise / fehlgeschlagen)	fehlgeschlagen
Anzahl Versuche	3
Manueller Eingriff nötig? (ja / nein)	nein
Falsche Aktion ausgeführt? (ja / nein)	nein, es wurde garnichts ausgeführt
Auffällige Wartezeit spürbar? (ja / nein)	ja, über 2sek, genau 10sek
Musste TP näher an den Bildschirm / Lupe neu positionieren, um Ergebnis zu prüfen?	ja
Beobachtungen der durchführenden Person	Endlosschleife der Generierung eines nicht existierenden Selektors, also die Felder werden nicht gefunden und das lokale Aktionsmodel panicked

Telemetrie zu dieser Aufgabe

Metrik	Wert
Telemetrie-Datei-ID	lava-telemetry-anonymized-tests-tp1.json
<code>pipeline_total</code> durationMs	74363 (Versuch 1, Feld befüllen) / 72177 (Versuch 2, inkl. 1 Replanning-Schritt)
<code>replanIterations</code>	1 (Versuch 1) / 2 (Versuch 2)
<code>actionCount</code>	1 (Versuch 1) / 3 (Versuch 2, kumuliert über beide Planungsschritte)
Deterministischer Fallback ausgelöst? (ja / nein)	nein
Subjektive Wartezeit (Kap. 6) vs. gemessene <code>pipeline_total</code> — Abweichung?	TP schätzte ca. 10 Sek., gemessen wurden 74,4 Sek. bzw. 72,2 Sek.

4.3 Aufgabe 2: Websuche

***Gesprochener Auftrag an TP:** Suche auf Google nach „Borussia Dortmund“ und öffne das erste passende Ergebnis.*

Teilkriterien (intendierter Endzustand):

1. Suchbegriff „Borussia Dortmund“ wurde korrekt in das Suchfeld eingetragen.
2. Die Suche wurde gestartet (Ergebnisliste erschienen), nicht nur eingetippt.
3. Das **erste passende** Ergebnis wurde geöffnet.

Feld	Angabe
Tatsächlich gesprochener Befehl	Gib in das Suchfeld ein, Borussia Dortmund, und führe die Suche durch.
Ergebnis (erfolgreich / teilweise / fehlgeschlagen)	erfolgreich
Anzahl Versuche	1
Manueller Eingriff nötig? (ja / nein)	nein
Falsche Aktion ausgeführt? (ja / nein)	ja, bei Google wurde noch Bilder angeklickt
Auffällige Wartezeit spürbar? (ja / nein)	ja, alles über 2sek ist lang
Musste TP näher an den Bildschirm / Lupe neu positionieren, um Ergebnis zu prüfen?	ja
Beobachtungen der durchführenden Person	/

Telemetrie zu dieser Aufgabe

Metrik	Wert
Telemetrie-Datei-ID	lava-telemetry-anonymized-tests-tp1.json
<code>pipeline_total</code> durationMs (Suche)	70432
<code>pipeline_total</code> durationMs (Ergebnis öffnen)	13426
<code>replanIterations</code> (jeweils)	1 / 1
<code>actionCount</code> (jeweils)	2 (fill+click) / 2 (click+fill)
Deterministischer Fallback ausgelöst? (ja / nein)	nein
Subjektive Wartezeit vs. gemessene <code>pipeline_total</code> — Abweichung?	TP nannte keinen genauen Wert, gemessen wurden 70,4 Sek. bzw. 13,4 Sek.

4.4 Aufgabe 3: Anmeldeformular

Gesprochener Auftrag an TP: Fülle das Anmeldeformular aus und sende es ab. (Formular-Seite/URL: Confluence Anmeldeseite)

Teilkriterien (intendierter Endzustand):

1. Alle Pflichtfelder wurden befüllt (Benutzername und Passwort, kein echtes Passwort).
2. Die eingetragenen Werte sind plausibel / wie von TP beabsichtigt.
3. Das Formular wurde abgesendet, nicht nur Felder gefüllt.

Feld	Angabe
Tatsächlich gesprochener Befehl	Trage als Benutzername P00.... und als Passwort 12345 und melde dich an.
Ergebnis (erfolgreich / teilweise / fehlgeschlagen)	teilweise
Anzahl Versuche	3
Manueller Eingriff nötig? (ja / nein)	nein
Falsche Aktion ausgeführt? (ja / nein)	ja, beim zweiten versuch den einloggen button drücken zu lassen wurde die suche geöffnet
Auffällige Wartezeit spürbar? (ja / nein)	ja, immernoch nach 3-4 Befehlen bei 10sek ca.
Musste TP näher an den Bildschirm / Lupe neu positionieren, um Ergebnis zu prüfen?	ja
Beobachtungen der durchführenden Person	Eingabe erfolgte, jedoch Zahlen wurde als Wörter ausgeschrieben

Telemetrie zu dieser Aufgabe

Metrik	Wert
Telemetrie-Datei-ID	lava-telemetry-anonymized-tests-tp1.json
<code>pipeline_total</code> durationMs	17196 (Versuch 1, Felder befüllen) / 20183 (Versuch 2, Login-Klick → falsche Aktion) / 20665 (Versuch 3, Login-Klick korrekt)
<code>replanIterations</code>	1 / 1 / 1
<code>actionCount</code>	2 / 1 / 1
Deterministischer Fallback ausgelöst? (ja / nein)	nein
Subjektive Wartezeit vs. gemessene <code>pipeline_total</code> — Abweichung?	TP schätzte ca. 10 Sek. je Befehl; gemessen wurden 17,2–20,7 Sek. — etwa doppelt so hoch wie geschätzt

4.5 Aufgabe 4: Amazon Produkt suchen und in Warenkorb legen

Gesprochener Auftrag an TP: Suche bei Amazon nach „Samsung Galaxy S26“ und lege es in den Warenkorb.

Teilkriterien (intendierter Endzustand):

1. Suchbegriff wurde korrekt eingetragen, Suche gestartet.
2. Das richtige Produkt (bzw. eine plausible Auswahl daraus) wurde geöffnet.
3. „In den Einkaufswagen“ wurde ausgelöst und der Artikel liegt im Warenkorb.

Feld	Angabe
Tatsächlich gesprochener Befehl	Trage in die Suchleiste Samsung Galaxy S26 ein und starte die Suche.
Ergebnis (erfolgreich / teilweise / fehlgeschlagen)	teilweise
Anzahl Versuche	2
Manueller Eingriff nötig? (ja / nein)	nein
Falsche Aktion ausgeführt? (ja / nein)	ja, anstatt den Such Button wurde ein Produkt auf der Startseite geöffnet
Auffällige Wartezeit spürbar? (ja / nein)	ja, über 10sek
Musste TP näher an den Bildschirm / Lupe neu positionieren, um Ergebnis zu prüfen?	ja
Beobachtungen der durchführenden Person	Anstatt der Suchbutton wird ein zufälliges Produkt ausgewählt.

Telemetrie zu dieser Aufgabe

Metrik	Wert
Telemetrie-Datei-ID	lava-telemetry-anonymized-tests-tp1.json
<code>pipeline_total</code> durationMs je Schritt	73808 (Versuch 1, Suche) / 78259 (Versuch 2, nach Fehlklick erneuter Versuch)
<code>replanIterations</code> je Schritt	1 / 1
Cap erreicht (5)? (ja / nein)	nein
<code>actionCount</code> gesamt	4 (2 + 2)
Deterministischer Fallback ausgelöst? (ja / nein)	nein
Subjektive Wartezeit vs. gemessene <code>pipeline_total</code> — Abweichung?	TP empfand „über 10 Sek.“, gemessen wurden 73,8 Sek. bzw. 78,3 Sek.

5. Spezifisch beobachtete Barrieren / Erleichterungen

Konkrete Momente festhalten, in denen die Sehbehinderung die Bedienung sichtbar erschwert **oder** die Sprachsteuerung sichtbar erleichtert hat (für die Thesis besonders relevant — bitte möglichst wörtlich/konkret beschreiben, nicht nur zusammenfassen):

Erschwernisse durch Bildschirmlupe (z. B. Ausschnitt zu klein, um Status/Feedback der Extension zu sehen):

Nein

Erschwernisse durch geringen Bildschirmabstand (z. B. Kopf/Körperhaltung, Mikrofonabstand verändert sich):

Nein garnicht

Momente, in denen die Sprachsteuerung einen Vorteil gegenüber Maus/Tastatur bot:

Durch das Requirement dass die Prüfung mit der Maus mindestens gemacht werden muss, nein.

Stellen, an denen visuelles Feedback der Extension (Status, Fehlermeldung, Hervorhebung) nicht wahrnehmbar war oder zu spät bemerkt wurde:

Passt alles!

6. Feedback der Testperson

Bitte möglichst wörtlich mitschreiben, Zitate sind für die Diskussion der Thesis wertvoller als paraphrasierte Zusammenfassungen.

Was hat gut funktioniert?

Darstellung des Transkripts (Rückmeldung), Sichtbarkeit vom aktuellen Status über unter anderem Farben

Was war frustrierend oder hat nicht funktioniert?

Dauer ist aktuell noch zu lang (Aktionsplanung).

Würde die Aufgabe ohne die Extension leichter oder schwerer werden?

Aktuell eher schwerer, dadurch, dass es noch nicht gut genug funktioniert.

Würde TP1 die Extension im Alltag nutzen? Warum/warum nicht?

An sich gern, aber die Qualität ist noch nicht gut genug.

Sonstige spontane Kommentare während des Tests:

Idee: Handhabung von Sprachbefehlen für die Eingabe von Adressdaten mit der genauen Aufteilung von Zahlen und Buchstaben.

7. Abschlussinterview

Frage	Antwort der Testperson
Wie einfach war es, die Extension zu bedienen? (1 = sehr schwer, 5 = sehr einfach)	4
Wie verständlich war das visuelle Feedback der Extension? (1–5)	5
Wie sehr hat die Extension die Aufgabe im Vergleich zur gewohnten Bedienung erleichtert? (1–5)	3
Wie vertrauenswürdig fanden Sie die ausgeführten Aktionen? (1–5)	2
Was würden Sie an der Extension ändern, um sie für Ihre Bedürfnisse besser nutzbar zu machen?	Erstmal muss die Geschwindigkeit besser werden
Gibt es Situationen im Alltag, in denen Sie sich diese Extension besonders wünschen würden?	Große Formulare mit vielen Feldern oder Email diktieren

8. Einschätzung der durchführenden Person

Aufgetretene technische Probleme (unabhängig von der Sehbehinderung):

Ausversehentliches Ablehnen auf der Berechtigungsseite und Laptop kam nicht an seine Grenzen und schränkte keine andere Programme/Prozesse ein, aber man hatte das Gefühl mehr Leistung für mehr Tokens/sek beim Aktionsmodell wären auch gut gewesen.

Auffälligkeiten, die auf einen spezifischen Zusammenhang mit der Sehbehinderung hindeuten (vs. allgemeine UX-Probleme):

Nein

Vorschläge für Verbesserungen, die aus diesem Test hervorgehen:

Automatisches Aufnahme beenden, nachdem man aufgehört hat zu sprechen.

Ehrenwörtliche Erklärungen

Hiermit versichere ich, dass ich die vorliegende Arbeit in allen Teilen selbstständig angefertigt und keine anderen, als die in der Arbeit angegebenen Quellen und Hilfsmittel benutzt habe. Sämtliche wörtlichen oder sinngemäßen Übernahmen und Zitate, sowie alle Abschnitte, die mithilfe von KI-basierten Tools entworfen, verfasst und/oder bearbeitet wurden, sind kenntlich gemacht und nachgewiesen.

Im Anhang meiner Arbeit habe ich sämtliche KI-basierte Hilfsmittel angegeben. Diese sind mit Produktnamen und formulierten Eingaben (Prompts) in einem KI-Verzeichnis ausgewiesen.

Ich bin mir bewusst, dass die Verwendung von Texten oder anderen Inhalten und Produkten, die durch KI-basierte Tools generiert wurden, keine Garantie für deren Qualität darstellt. Ich verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst und trage die Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Berlin, 05.08.2026

Ort, Datum

Theo Leuthardt